

AI-powered Job Search

i want to make an search api for the jobs and i want to make this for an platform the backend and the api and the frontend search so i want it to get all the things from the job sites and top jobs and the things will be shown to the user after the search so i want to make this things proper and i want to make sure this works properly

search like the linkedin google jobs and the glassdoor so i want to make sure this works and it should have multiple things support and professional like the linkedin search and the data will get from all the available apis and the things

based on cv/resume jobs suggestions and also there will be an based on user profile and the most best option for the user based on the user profile or resume/cv and all the best things an jobs search can have all the parameters and the options all the options

and if you have more suggestions then enlighten me please

<https://github.com/AnumQ/find-jobs.git>

<https://github.com/MadsLorentzen/ai-job-search.git>

<https://github.com/santifer/career-ops.git>

<https://github.com/maddykws/AI-Powered-Job-Search-Tool.git>

do you understand this ??

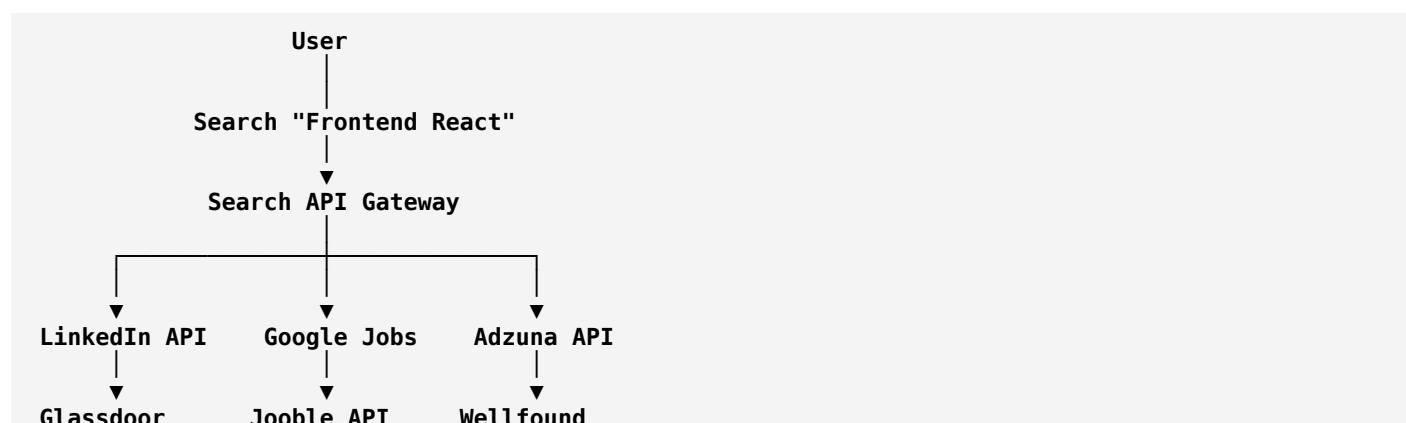
Show moreShow less

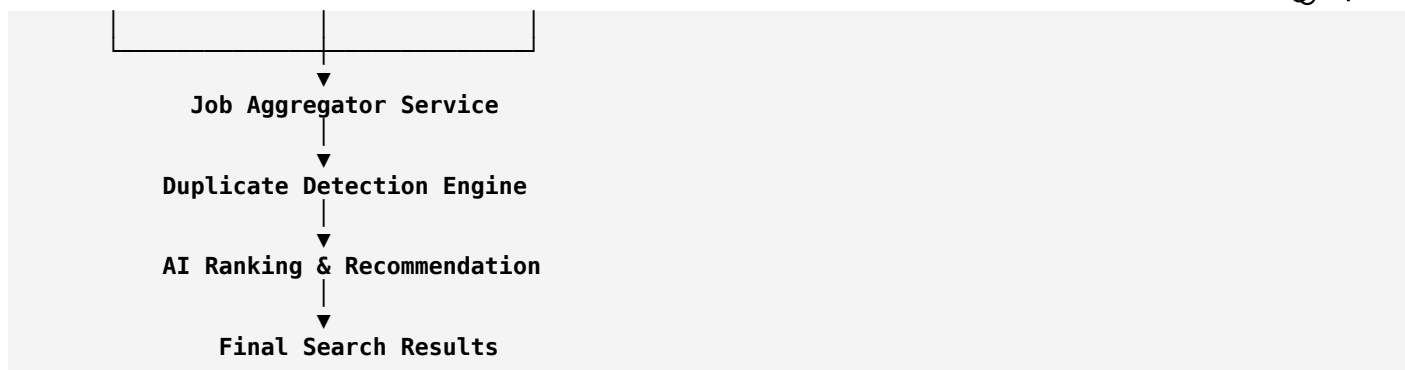
Yes. I understand exactly what you're trying to build.

What you're describing is **not just a job search page**. You're trying to build an **AI-powered Job Discovery Platform** similar to the search experience of LinkedIn Jobs + Google Jobs + Indeed + Glassdoor + Wellfound + ZipRecruiter, but with AI personalization.

The biggest mistake most developers make is trying to scrape LinkedIn or directly query job boards from the frontend. Instead, think of it as a **Job Search Engine** with multiple layers.

The Vision





This is how Google Jobs works internally.

Core Modules

I'd split it into independent services.

- Search Service
- Recommendation Service
- Resume Matching Service
- Saved Jobs Service
- Recent Searches
- Trending Jobs
- Company Service
- Salary Insights
- Job Alert Service
- Analytics Service
- AI Recommendation Service

Search Flow

User searches

React Developer
Mumbai
Remote
5 years
₹20 LPA

↓

Backend receives request

↓

Parallel API Calls

Google Jobs

LinkedIn

Jooble
Adzuna
USAJobs
Remotive
RemoteOK
Wellfound
Greenhouse
Lever
Ashby
Indeed (if available)
Glassdoor (if licensed)
CareerJet
JSearch API
RapidAPI Providers

↓
 Normalize every response
 ↓
 Remove duplicates
 ↓
 Rank
 ↓
 Return

Unified Job Model

Every provider returns different JSON.

Your backend should convert everything into one schema.

TypeScript
Job
id
title
company
logo
description
salary
currency
location

country
city
experience
skills
employmentType
remoteType
postedAt
applyUrl
companyWebsite
source
sourceLogo
sourcePriority
matchScore
aiScore
resumeScore
profileScore
benefits
visaSponsored
relocation
technologies
industry
education
certifications
companyRating
companyReviews
companySize
workMode
language
applicationDeadline
createdAt

Every provider maps into this.

Search Parameters

Not just keyword.

Something like

query
location
country
city
remote
hybrid
onsite
salaryMin
salaryMax
experienceMin
experienceMax
skills
company
industry
employmentType
contractType
education
certification
postedWithin
sort
page
limit
language
visa
relocation
benefits
companyRating
easyApply
topCompanies
startupOnly
fortune500
government
internship
fresher
seniorLevel

This is similar to LinkedIn.

Sorting

Relevance
Newest
Salary High
Salary Low
AI Match
Resume Match
Company Rating
Easy Apply
Trending
Most Applied
Most Viewed
Recommended
Recently Updated

AI Resume Matching

User uploads resume.

↓

Extract

Skills
Experience
Education
Projects
Achievements
Certificates
Technologies
Languages

↓

Create embeddings

↓

Search jobs

↓

Calculate

Skill Match

Experience Match
Education Match
Salary Match
Location Match
Industry Match



Return

92% Match
87% Match
76% Match

AI Profile Matching

Instead of resume only

Use

Profile
Saved Jobs
Viewed Jobs
Ignored Jobs
Applied Jobs
Preferred Salary
Preferred Cities
Remote Preference
Experience
Companies Worked
Education
Skills
Interests

Then personalize.

Netflix recommendation system but for jobs.

AI Ranking

Imagine 500 jobs found.

Don't return randomly.

Rank

Resume Similarity

Skill Match

Location Match

Salary Match

Freshness

Popularity

Company Quality

AI Confidence

Remote Preference

Easy Apply

Verified Employer

Final score

AI Score

94.3

NLP Search

Instead of keyword search

Support

React jobs

Remote React jobs

Remote React jobs under 25 LPA

Frontend startup jobs

Companies using Next.js

Google level frontend jobs

Product based companies

High salary backend jobs

AI Engineer Remote

Data Scientist Bangalore

Jobs like Amazon

Jobs requiring Docker

Natural language.

Smart Filters

LinkedIn has around 15.

I'd build 50+.

Basic

- Remote
- Hybrid
- Onsite
- Internship
- Fresher
- Full Time
- Part Time

Salary

10L
15L
20L
30L
50L

Experience

0-1
1-3
3-5
5-8
10+

Company

Startup
MNC
Fortune 500
FAANG
Verified
Hiring Fast

AI

Best Match
Resume Match
Profile Match

Fast Response
High Interview Rate
Easy Apply
Skill Gap <20%

Search Suggestions

Like Google.

As user types

rea
↓
React
ReactJS
React Developer
React Native
React Remote
React NextJS
React TypeScript

Autocomplete.

Company Intelligence

Every company page

Rating
Employees
Industry
Funding
Tech Stack
Interview Experience
Open Jobs
Benefits
Website
LinkedIn
Glassdoor
Growth
Hiring Trends

Salary Insights

React Developer

Mumbai

Average

₹18 LPA

Range

₹10-32 LPA

Top Companies

Top Skills

Highest Paying

AI Resume Suggestions

Your resume matches 81%

Missing

Docker

AWS

Jest

Redux Toolkit

GraphQL

Add these skills to increase matches.

AI Job Explanation

Instead of

Match 82%

Explain

You match because

✓ React

✓ Next.js

✓ TypeScript

✓ REST API

✓ Redux

Missing

Docker

AWS

CI/CD

Estimated interview chance

78%

Alerts

Notify user

New React Jobs

Salary increased

Company hiring

Dream company posted

Remote jobs

Resume Match >90%

Saved Searches

React Remote

Saved

↓

Daily Alerts

Weekly Alerts

Instant Alerts

Analytics

Jobs Viewed

Applications

Interviews

Response Rate

Resume Score

Average Match

Average Salary

Top Skills

Backend Architecture

Gateway API

↓

Search Service

↓
Aggregation Service
↓
Normalization Service
↓
Ranking Service
↓
Recommendation Service
↓
Resume Parser
↓
Embedding Service
↓
Redis Cache
↓
PostgreSQL
↓
OpenSearch / Elasticsearch
↓
AI Service
↓
Notification Service

Search API Example

```
http
GET /jobs/search
```

Query parameters:

```
TypeScript
{
  query,
  location,
  salaryMin,
  salaryMax,
  experience,
  skills,
  remote,
  company,
  page,
  limit,
  sort,
  resumeId,
  profileId
}
```

Response:

```
TypeScript
{
  jobs: [...],
  total,
  page,
  facets,
  suggestions,
  aiRecommendations,
  trendingSearches,
  relatedSearches
}
```

Technology Stack

For a scalable platform, I'd recommend:

- **Frontend:** React/Next.js with server-side rendering for SEO and fast initial loads.
- **Backend:** NestJS (or Express/Fastify if you prefer) with a modular service architecture.
- **Database:** PostgreSQL for relational data and user information.
- **Search Engine:** Elasticsearch or OpenSearch for full-text search, faceted filtering, and autocomplete.
- **Cache:** Redis for caching search results, suggestions, and rate limiting.
- **Message Queue:** RabbitMQ or Kafka for background ingestion, indexing, and notifications.
- **Object Storage:** AWS S3 (or equivalent) for resumes and documents.
- **AI Services:** LLMs for recommendations, resume analysis, and natural-language search; embedding models with a vector database such as pgvector or Qdrant for semantic matching.

Important considerations about external job sources

One point is crucial: **LinkedIn Jobs, Google Jobs, and Glassdoor generally do not provide unrestricted public APIs for aggregating all of their job listings.** Their data is protected by terms of service, licensing, or partner agreements. Building a production system by scraping these sites can lead to legal and operational issues.

A robust approach is to combine:

- Official APIs from providers that allow aggregation (such as Adzuna, Jooble, USAJobs, Remotive, Greenhouse, Lever, Ashby, etc.).
- Company career pages through ATS integrations (Greenhouse, Lever, Ashby, Workday where permitted).
- Structured feeds from partners.
- Your own indexed job database with deduplication and enrichment.

This gives you a stable, scalable foundation while remaining compliant.

My biggest recommendation

Don't think of this as "**a search API.**"

Think of it as building an **AI-powered Job Search Engine** with four independent capabilities:

1. **Aggregation Engine** – Collects jobs from multiple approved sources.
2. **Search Engine** – Provides fast, intelligent, faceted, and semantic search.
3. **Recommendation Engine** – Personalizes results based on profiles, resumes, and behavior.
4. **AI Career Assistant** – Explains matches, identifies skill gaps, suggests improvements, and helps users discover better opportunities.

If you build these as separate modules, you'll end up with a platform that can compete in functionality with modern job discovery products rather than a simple job listing website.

list all the features we have and we can add into this

If the goal is to build one of the best AI-powered job platforms, I'd organize the features into modules. This makes the product scalable and easier to develop.

1. Smart Job Search

Basic Search

- Keyword Search
- Company Search
- Job Title Search
- Skills Search
- Technology Search
- Industry Search
- Natural Language Search
- Boolean Search (AND/OR/NOT)
- Voice Search
- AI Search

Advanced Search

- Multi-keyword Search
- Search by Resume
- Search by Profile
- Search by Experience
- Search by Education

- Search by Certification
 - Search by Company Size
 - Search by Salary
 - Search by Benefits
 - Search by Visa Sponsorship
 - Search by Relocation
 - Search by Work Authorization
 - Search by Language
 - Search by Security Clearance
-

2. Search Filters

Location

- Country
- State
- City
- Area
- Radius
- Remote
- Hybrid
- On-site
- Worldwide Remote

Job Type

- Full-time
- Part-time
- Internship
- Contract
- Freelance
- Temporary
- Apprenticeship
- Volunteer
- Seasonal

Experience

- Fresher

- Entry
- Junior
- Mid
- Senior
- Lead
- Principal
- Manager
- Director
- VP
- Executive

Salary

- Minimum Salary
- Maximum Salary
- Hourly
- Monthly
- Yearly
- Currency
- Equity Included
- Bonus Included

Company

- Startup
- Unicorn
- Fortune 500
- FAANG
- MNC
- Government
- NGO
- Verified Employer

Posting

- Last Hour
- Last 24 Hours
- Last 3 Days
- Last Week

- Last Month
- Custom Date

Benefits

- Health Insurance
 - Remote Work
 - Flexible Hours
 - Gym
 - Bonus
 - Stock Options
 - Learning Budget
 - Unlimited PTO
 - Visa Sponsorship
 - Relocation
-

3. AI Search

- Natural Language Search
- Semantic Search
- Intent Detection
- Skill Understanding
- Synonym Matching
- Job Similarity
- Company Similarity
- AI Query Expansion
- AI Ranking
- AI Explanation
- AI Search Suggestions
- AI Related Searches

Example:

"Remote React jobs paying above ₹20L with startups using Next.js"

4. Resume Analysis

- Upload Resume
- Resume Parsing

- Skill Extraction
 - Experience Extraction
 - Education Extraction
 - Project Extraction
 - Achievement Extraction
 - Certificate Extraction
 - Language Detection
 - Resume Score
 - ATS Score
 - Resume Completeness
 - Missing Skills
 - Improvement Suggestions
-

5. Profile Based Recommendations

- Based on Skills
 - Based on Experience
 - Based on Previous Jobs
 - Based on Saved Jobs
 - Based on Applied Jobs
 - Based on Preferred Salary
 - Based on Preferred Companies
 - Based on Preferred Location
 - Based on Interests
 - Based on Learning History
-

6. AI Matching Engine

Every job receives scores like:

- Resume Match
- Profile Match
- Skill Match
- Experience Match
- Education Match
- Project Match
- Industry Match

- Company Match
- Salary Match
- Location Match
- Career Growth Match
- Technology Match
- Soft Skill Match

Overall

94% Match

7. AI Explanations

Instead of

82% Match

Explain

You match because

- ✓ React
- ✓ TypeScript
- ✓ Next.js
- ✓ REST API

Missing

Docker
AWS
CI/CD

Estimated Interview Chance

78%

8. Job Sources

Aggregate jobs from

- Company Career Pages
- ATS Platforms (Greenhouse, Lever, Ashby, Workday where supported)
- Adzuna
- Jooble
- USAJobs
- Remotive
- RemoteOK
- Wellfound
- CareerJet

- RapidAPI providers
 - Internal Job Database
 - Employer Portal
-

9. Search Suggestions

Autocomplete

Trending Searches

Popular Searches

Recent Searches

Related Searches

AI Suggestions

Skill Suggestions

Company Suggestions

Role Suggestions

10. Saved Items

- Saved Jobs
 - Saved Companies
 - Saved Searches
 - Saved Filters
 - Favorite Recruiters
 - Favorite Industries
-

11. Job Alerts

Email Alerts

SMS Alerts

Push Notifications

WhatsApp Alerts

Instant Alerts

Daily Digest

Weekly Digest

Monthly Digest

AI Recommended Alerts

12. Company Pages

- Company Overview
 - Ratings
 - Reviews
 - Salaries
 - Interview Experience
 - Benefits
 - Culture
 - Tech Stack
 - Open Positions
 - Hiring Trends
 - Funding
 - Employee Count
-

13. Salary Intelligence

- Average Salary
 - Salary Range
 - Top Paying Companies
 - Salary Growth
 - Salary by Experience
 - Salary by Location
 - Salary by Skill
-

14. Career Insights

Career Path

Growth Prediction

Promotion Timeline

Demand Forecast

Future Skills

Trending Technologies

Industry Outlook

Learning Roadmap

15. Skill Gap Analysis

Missing Skills

Trending Skills

Recommended Courses

Certification Suggestions

Learning Roadmap

Difficulty Level

Estimated Learning Time

16. AI Career Coach

Resume Review

Career Advice

Interview Preparation

Salary Negotiation

Offer Comparison

Career Switch Suggestions

Promotion Guidance

Skill Planning

17. Job Comparison

Compare

- Salary
 - Benefits
 - Location
 - Company Rating
 - Tech Stack
 - Growth
 - Work-life Balance
 - Commute
 - Match Score
-

18. Employer Features

Post Jobs
Manage Jobs
View Applicants
AI Resume Ranking
Candidate Search
Interview Scheduling
Offer Management
Analytics Dashboard

19. Applicant Tracking

Application Status
Interview Status
Offer Status
Rejected
Accepted
Withdrawn
Application Timeline

20. Analytics Dashboard

Applications Sent
Interview Rate
Offer Rate
Resume Views
Recruiter Views
Average Match Score
Profile Strength
Salary Growth
Top Skills
Skill Demand

21. Social Features

Follow Companies
Follow Recruiters

Follow Industries
Share Jobs
Refer Friends
Community Discussions
Career Groups

22. Security

Verified Employers
Verified Recruiters
Spam Detection
Fraud Detection
Duplicate Detection
Fake Job Detection
Scam Warning

23. Admin Features

Manage Jobs
Manage Employers
Manage Users
Moderation
Reports
Analytics
API Monitoring
Search Analytics
Revenue Dashboard

24. Search Performance

Redis Cache
Elastic/OpenSearch
AI Ranking
Faceted Search
Infinite Scroll
Cursor Pagination

Search History
Search Analytics
Real-time Suggestions
Spell Correction
Did You Mean
Synonym Dictionary
Typo Tolerance

25. AI Features

Resume Embeddings
Job Embeddings
Semantic Search
Hybrid Search
Vector Search
RAG (Retrieval-Augmented Generation)
LLM Recommendations
AI Summaries
AI Skill Extraction
AI Resume Builder
AI Cover Letter Generator
AI Interview Simulator
AI Mock Interviews
AI Salary Prediction
AI Company Insights
AI Career Roadmap
AI Job Description Summaries
AI Recruiter Assistant

26. Future Features

- One-click apply across supported platforms
- Browser extension to save jobs from any website
- Chrome extension for AI job matching while browsing
- Calendar integration for interviews
- AI interview transcription and feedback

- Referral marketplace
- Alumni and network-based referrals
- Recruiter messaging
- Portfolio/GitHub analysis
- Coding profile integration (LeetCode, HackerRank, Codeforces)
- GitHub project analysis for skill extraction
- Personalized learning recommendations
- Labor market and hiring trend analytics
- Multi-language support
- Global salary normalization
- Accessibility features
- Offline/mobile-first experience

Long-term vision

Instead of positioning the platform as "**a job portal**," aim to build an **AI Career Operating System** with six core pillars:

1. **Job Discovery** – Aggregates and searches opportunities from multiple legitimate sources.
2. **Career Intelligence** – Salary insights, market trends, company intelligence, and skill demand.
3. **AI Personalization** – Resume parsing, profile understanding, semantic search, and recommendations.
4. **Application Management** – Saved jobs, application tracking, alerts, interview scheduling, and offer comparison.
5. **Career Growth** – Skill-gap analysis, learning roadmaps, interview preparation, and career coaching.
6. **Employer Platform** – Recruitment tools, AI candidate matching, analytics, and hiring workflows.

This architecture gives you a platform that can continue to grow without becoming a monolithic job board.

One-click apply across supported platforms
 Portfolio/GitHub analysis
 GitHub project analysis for skill extraction
 Multi-language support
 Global salary normalization
 mobile-first experience

add this to the current thing
 Show moreShow less

Absolutely. These features fit very well into the roadmap. I'd expand them slightly so they're production-ready.

27. One-Click Apply & Application Automation

One-Click Apply

- One-click apply on supported job providers
- Auto-fill application forms
- Resume selection
- Cover letter selection
- AI-generated cover letters
- Portfolio attachment
- Multiple resume versions
- Custom application answers
- Track application status
- Application history
- Retry failed applications
- Bulk apply (supported platforms)
- Save incomplete applications
- Easy Apply filter
- Auto-follow application links
- Application confirmation tracking

Smart Apply Assistant

- ATS compatibility check before applying
- Resume optimization for the selected job
- AI-generated custom resume
- AI-generated custom cover letter
- Missing information detection
- Estimated interview probability
- AI application quality score

28. Portfolio & Developer Profile Analysis

Portfolio Analysis

- Portfolio URL upload

- Portfolio quality score
- Project relevance analysis
- Technology detection
- UI/UX quality analysis
- Accessibility analysis
- SEO analysis
- Performance analysis
- Mobile responsiveness analysis
- Live project verification
- Broken link detection
- Domain health check

Supported Platforms

- GitHub
- GitLab
- Bitbucket
- Personal Website
- Behance
- Dribbble
- Figma
- CodePen
- Stack Overflow
- Medium
- Dev.to
- Kaggle
- LeetCode
- HackerRank
- Codeforces
- TopCoder

29. GitHub Intelligence & Skill Extraction

Repository Analysis

- Repository scanning
- Programming language detection

- Framework detection
- Library detection
- Architecture detection
- Database detection
- Cloud technology detection
- DevOps tool detection
- Testing framework detection

AI Skill Extraction

- Extract technical skills
- Extract soft skills (from contributions)
- Experience estimation
- Technology proficiency score
- Framework proficiency score
- Skill confidence score
- Skill timeline
- Skill growth over time

Code Quality Analysis

- Repository health
- Commit frequency
- Contribution consistency
- Code complexity
- Documentation quality
- README quality
- Open-source contributions
- Pull request history
- Issue participation
- Code review activity

Project Intelligence

- Best projects
- Featured projects
- Similar technologies
- Real-world experience validation
- Enterprise project detection

- Personal project detection
- Team collaboration analysis

GitHub AI Match

- GitHub Match Score
 - Technology Match
 - Project Match
 - Open Source Score
 - Engineering Profile Score
 - Recruiter Readiness Score
-

30. Multi-Language Support

User Interface

- Multi-language UI
- RTL language support
- Dynamic language switching
- Localized formatting
- Regional date & time formats

Search

- Multi-language job search
- Cross-language search
- AI translation of search queries
- Language-specific autocomplete
- Synonym matching across languages

Job Content

- Translate job descriptions
- Translate company information
- Translate benefits
- Translate requirements
- Translate recruiter messages

Resume

- Resume translation

- Cover letter translation
- AI multilingual resume generation
- Language proficiency detection

Communication

- AI recruiter message translation
 - Candidate-recruiter multilingual chat
 - Email translation
 - Notification localization
-

31. Global Salary Intelligence

Salary Normalization

- Currency conversion
- Purchasing Power Parity (PPP)
- Cost of Living adjustment
- Tax estimation
- Net salary estimation
- Inflation adjustment

Salary Comparison

- Compare salaries across countries
- Compare cities
- Compare industries
- Compare companies
- Compare experience levels

Compensation Breakdown

- Base salary
- Bonus
- Equity
- Stock options
- Benefits value
- Total compensation
- Estimated take-home salary

Salary Insights

- Salary growth prediction
 - Market demand
 - Future salary projection
 - Salary negotiation suggestions
 - Fair salary estimation
-

32. Mobile-First Experience

Responsive Platform

- Fully responsive UI
- Progressive Web App (PWA)
- Offline support
- Fast loading
- Adaptive layouts
- Gesture navigation

Mobile Features

- Swipe to save
- Swipe to apply
- Voice search
- Camera resume upload
- Document scanning
- Push notifications
- Biometric login
- Deep linking

Mobile Productivity

- Resume upload from device
- Cloud storage integration
- Calendar integration
- Interview reminders
- GPS-based nearby jobs
- Quick Apply
- Share jobs
- Save offline

Performance

- Lazy loading
 - Infinite scrolling
 - Image optimization
 - Cached search
 - Background synchronization
 - Battery-efficient updates
-

33. Developer & Technical Profile (New Module)

This is something most job platforms don't do well and can become a major differentiator.

Technical Profile

- GitHub Profile Analysis
 - GitLab Profile Analysis
 - Stack Overflow Reputation
 - LeetCode Statistics
 - HackerRank Badges
 - Codeforces Rating
 - Kaggle Profile
 - NPM Package Analysis
 - Open Source Contributions
 - Technical Blog Analysis
 - Portfolio Verification
 - Live Project Verification
 - AI-generated Developer Skill Matrix
 - Engineering Maturity Score
 - Recruiter Readiness Score
 - Technical Portfolio Score
 - Open Source Impact Score
 - Community Contribution Score
-

Overall Platform Vision

After adding these modules, your platform evolves from a **job board** into a comprehensive **AI Career Intelligence Platform**. It can be organized into the following major domains:

- 1. Job Discovery & Search** – Intelligent, multi-source job aggregation with advanced search and filtering.
- 2. AI Personalization** – Resume parsing, profile understanding, semantic search, recommendations, and matching.
- 3. Career Intelligence** – Salary insights, company intelligence, market trends, and career growth analytics.
- 4. Developer Intelligence** – GitHub and portfolio analysis, coding profile integration, and technical skill extraction.
- 5. Application Management** – One-click apply, AI-assisted applications, tracking, alerts, and interview scheduling.
- 6. Employer Platform** – Job posting, AI candidate matching, recruitment workflows, and hiring analytics.
- 7. Global Experience** – Multi-language support, localized search, global salary normalization, and mobile-first access.

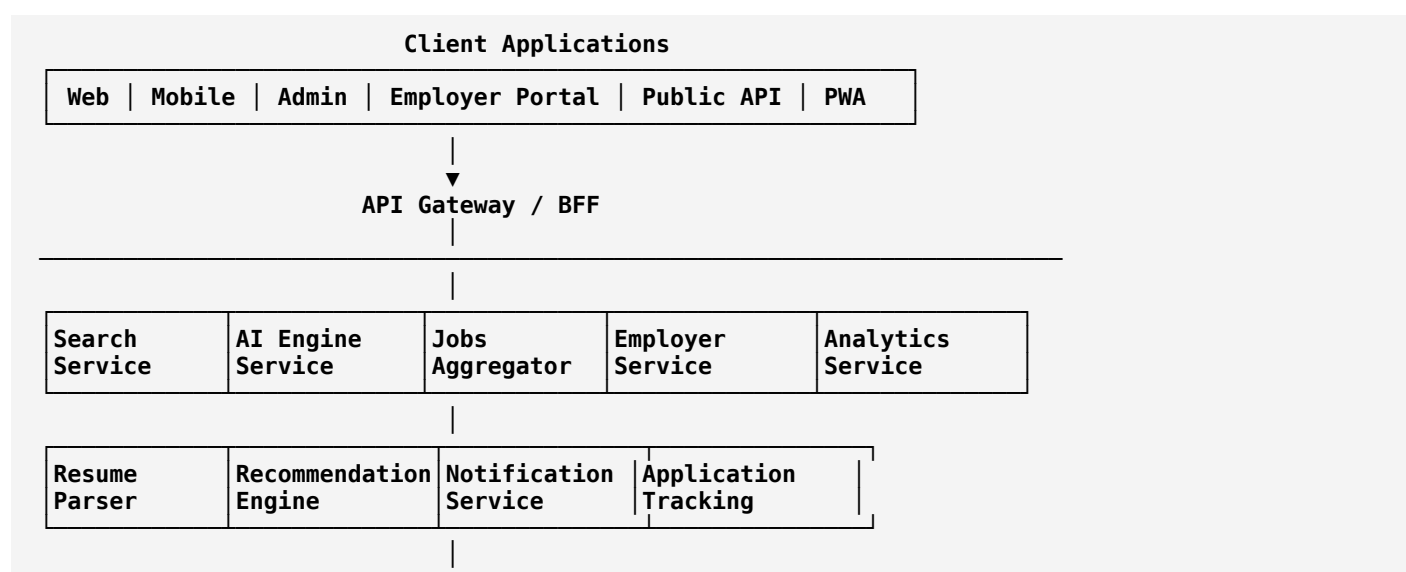
With these additions, the platform would cover virtually every stage of a candidate's career journey—from discovering opportunities to applying intelligently, improving skills, and managing long-term career growth.

now give me this as an full stack engineer will look into this

AI Career Platform (Full Stack System Design & Feature Specification)

Vision: Build an AI-powered Career Intelligence Platform that aggregates jobs from multiple providers, delivers highly personalized search and recommendations, and provides end-to-end career management for candidates while offering intelligent hiring tools for employers.

1. System Architecture



Postgres	Redis	ElasticSearch / OpenSearch	Vector DB (pgvector/Qdrant)
----------	-------	----------------------------	-----------------------------

2. Core Services

Authentication Service

- Authentication
- Authorization
- OAuth Login
- Social Login
- Session Management
- RBAC
- MFA
- JWT
- API Keys

User Service

- User Profile
- Preferences
- Resume Management
- Portfolio Management
- Skills
- Certifications
- Experience
- Education
- Languages
- Career Goals

Search Service

Responsible for all search operations.

Features

- Keyword Search

- Semantic Search
 - AI Search
 - Natural Language Search
 - Boolean Search
 - Voice Search
 - Multi-source Search
 - Hybrid Search
 - Vector Search
 - Faceted Search
 - Fuzzy Search
 - Synonym Search
 - Typo Tolerance
 - Auto Suggestions
 - Related Searches
 - Trending Searches
 - Search History
 - Recent Searches
 - Search Analytics
-

3. Job Aggregation Service

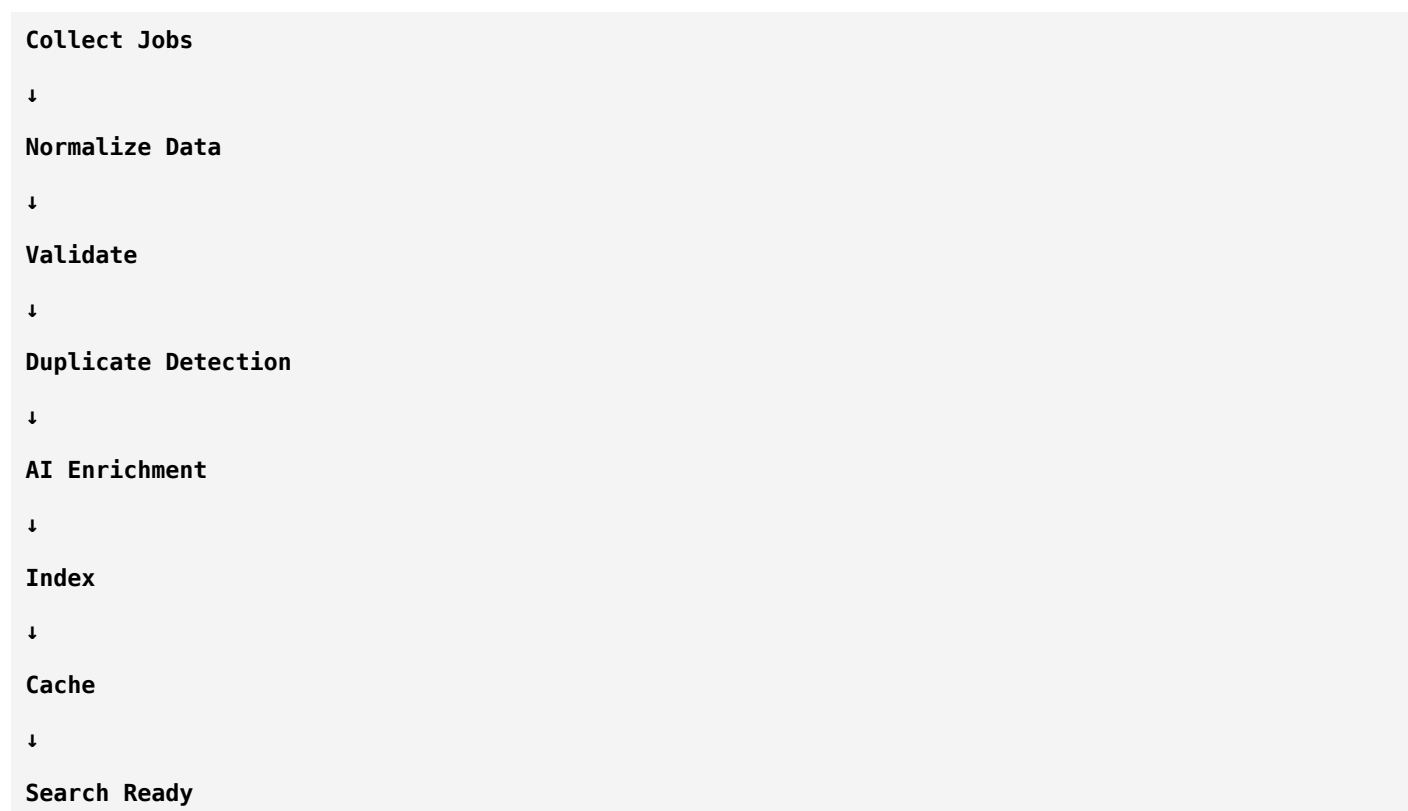
Responsible for collecting jobs.

Sources

- ATS Platforms
 - Greenhouse
 - Lever
 - Ashby
 - Workday (where permitted)
- Public APIs
 - Adzuna
 - Jooble
 - USAJobs
 - Remotive
 - RemoteOK

- CareerJet
- RapidAPI Job Providers
- Company Career Pages
- Employer Portal
- Internal Jobs

Processing Pipeline



4. Search Filters

Query

- Keywords
- Skills
- Company
- Industry
- Technologies

Location

- Country

- State
- City
- Radius
- Remote
- Hybrid
- Onsite

Employment

- Full Time
- Part Time
- Internship
- Contract
- Freelance
- Temporary

Salary

- Minimum
- Maximum
- Currency
- Hourly
- Monthly
- Annual

Experience

- Fresher
- Entry
- Mid
- Senior
- Lead
- Director

Company

- Startup

- Unicorn
- Enterprise
- Government
- NGO
- Fortune 500
- Verified

Job Details

- Visa Sponsorship
 - Relocation
 - Education
 - Certifications
 - Languages
 - Benefits
 - Easy Apply
-

5. AI Search

Natural Language Queries

Examples

```
React jobs in Bangalore
Remote Next.js jobs above ₹20L
Backend jobs with visa sponsorship
Companies hiring React developers
Startup jobs using AI
Product-based companies hiring freshers
```

AI Capabilities

- Intent Detection
- Entity Recognition
- Skill Extraction
- Semantic Matching
- Query Expansion

- AI Ranking
 - Search Personalization
-

6. Resume Intelligence

Resume Upload

- PDF
 - DOCX
 - DOC
-

AI Resume Parsing

Extract

- Skills
 - Technologies
 - Experience
 - Education
 - Projects
 - Certifications
 - Languages
 - Achievements
-

Resume Analysis

- Resume Score
 - ATS Score
 - Missing Skills
 - Resume Suggestions
 - Grammar Check
 - AI Improvements
 - Job Compatibility
-

7. Profile Intelligence

Profile Analysis

- Skills
 - Experience
 - Preferred Salary
 - Preferred Cities
 - Preferred Companies
 - Career Interests
 - Learning Progress
-

AI Recommendation

Recommend jobs using

- Resume
 - User Profile
 - Search History
 - Saved Jobs
 - Applied Jobs
 - Portfolio
 - GitHub
 - Skills
 - Career Goals
-

8. AI Matching Engine

Calculate

- Resume Match
- Profile Match
- Skill Match
- Experience Match
- Education Match
- Project Match
- Technology Match
- Salary Match
- Location Match

- Industry Match

Generate

Overall Match Score

0–100%

Explain

Matched Skills

Missing Skills

Reasons

Suggestions

9. Job Search Result

Every Job Contains

Job

Company

Logo

Salary

Currency

Location

Remote Status

Employment Type

Experience

Description

Benefits

Skills

Technologies

Recruiter

Apply URL

Easy Apply

Match Score

AI Score

Company Rating

Reviews

Posted Date

Source

Application Deadline

10. Recommendation Engine

Recommendations based on

- Resume
 - User Profile
 - Skills
 - Search Behavior
 - Saved Jobs
 - Viewed Jobs
 - Applied Jobs
 - Trending Jobs
 - Market Demand
 - Career Goals
-

11. Job Alerts

- Email
 - SMS
 - Push
 - WhatsApp
 - Daily Digest
 - Weekly Digest
 - AI Recommendations
 - Salary Alerts
 - Dream Company Alerts
-

12. Company Intelligence

Every Company

- About
- Industry
- Size
- Funding
- Benefits
- Reviews
- Salaries

- Tech Stack
 - Hiring Trends
 - Open Jobs
 - Growth Rate
-

13. Salary Intelligence

- Market Salary
 - Company Salary
 - City Salary
 - Country Salary
 - Salary Trends
 - Cost of Living
 - Tax Estimation
 - PPP Adjustment
 - Currency Conversion
 - Total Compensation
-

14. Portfolio Intelligence

Supported Platforms

- GitHub
- GitLab
- Bitbucket
- Personal Website
- Behance
- Dribbble
- Figma
- CodePen
- Stack Overflow
- Medium
- Dev.to
- Kaggle
- LeetCode
- HackerRank

Analysis

- Portfolio Score
 - UI Quality
 - Accessibility
 - SEO
 - Performance
 - Live Links
 - Mobile Responsiveness
 - Project Quality
-

15. GitHub Intelligence

Repository Analysis

- Languages
- Frameworks
- Libraries
- Architecture
- Cloud
- DevOps
- Databases
- Testing

AI Extracts

- Skills
- Technologies
- Experience
- Framework Proficiency
- Contribution Quality
- Commit Consistency
- Open Source Activity

Scores

- Developer Score
- GitHub Score
- Engineering Score
- Recruiter Readiness

- Project Quality

16. One-Click Apply

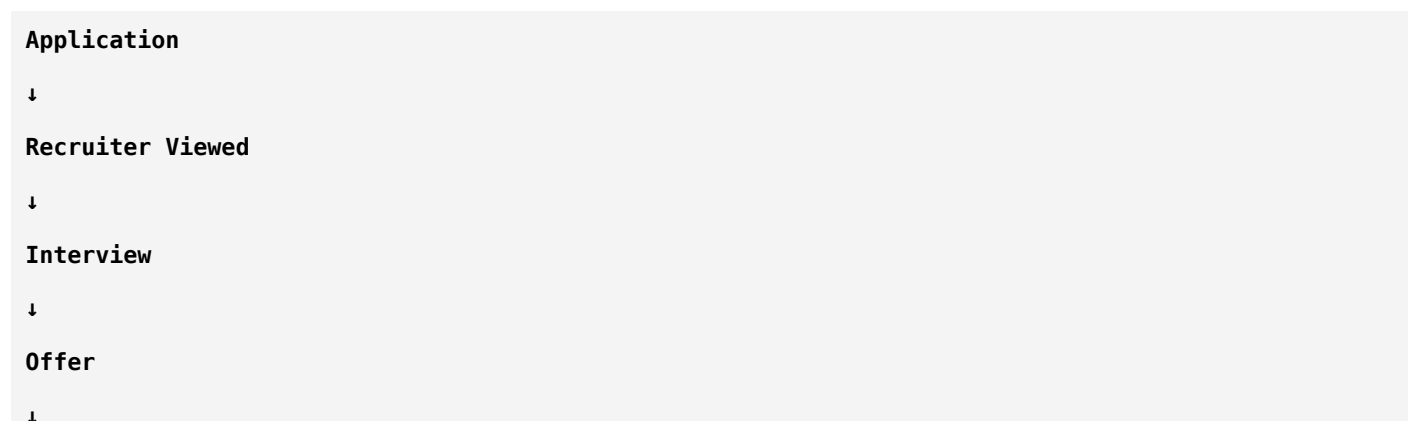
- Easy Apply
- Resume Selection
- Cover Letter Selection
- AI Cover Letter
- Portfolio Attachment
- Multiple Resume Versions
- Auto Form Filling
- Application Tracking
- AI Resume Optimization

17. Application Tracker

Track

- Applied
- Under Review
- Screening
- Interview
- Assessment
- HR Round
- Technical Round
- Offer
- Accepted
- Rejected

Timeline



18. AI Career Coach

- Resume Review
 - Career Advice
 - Salary Negotiation
 - Interview Preparation
 - Mock Interviews
 - Skill Gap Analysis
 - Learning Roadmap
 - Career Switch Guidance
-

19. Multi-Language Support

Frontend

- Multi-language UI
- RTL Support
- Localization
- Regional Formatting

Backend

- Multi-language Search
- Translation
- Localized Notifications
- Localized Emails

AI

- Resume Translation
 - Job Translation
 - Recruiter Message Translation
-

20. Mobile-First Platform

- Fully Responsive
- PWA
- Offline Mode

- Camera Resume Upload
 - Voice Search
 - Push Notifications
 - Biometric Login
 - Deep Linking
 - Swipe Actions
 - GPS Nearby Jobs
 - Infinite Scroll
-

21. Employer Platform

- Employer Dashboard
 - Job Posting
 - Applicant Tracking
 - Candidate Search
 - AI Candidate Ranking
 - Interview Scheduling
 - Offer Management
 - Team Collaboration
 - Analytics
-

22. Admin Platform

- User Management
 - Employer Management
 - Job Moderation
 - Fraud Detection
 - Reports
 - Search Analytics
 - Revenue Dashboard
 - API Monitoring
 - Audit Logs
-

23. Security

- RBAC

- JWT
 - OAuth
 - MFA
 - Rate Limiting
 - CAPTCHA
 - Spam Detection
 - Fake Job Detection
 - Duplicate Detection
 - Secure File Storage
 - Audit Logging
-

24. Analytics

User

- Search Analytics
- Resume Views
- Recruiter Views
- Match Scores
- Application Success Rate

Employer

- Job Views
- Applications
- Conversion Rate
- Candidate Funnel

Platform

- Search Trends
 - Popular Skills
 - Hiring Trends
 - Salary Trends
 - API Metrics
-

25. AI Services

- Semantic Search
- Vector Embeddings

- Resume Embeddings
- Job Embeddings
- RAG-based Recommendations
- AI Ranking
- AI Summaries
- AI Resume Builder
- AI Cover Letter Generator
- AI Interview Simulator
- AI Skill Gap Detection
- AI Career Roadmap
- AI Salary Prediction
- AI Job Description Simplification

26. Performance & Scalability

- Redis Caching
- Elasticsearch/OpenSearch
- CDN
- Background Workers
- Queue Processing (BullMQ/RabbitMQ/Kafka)
- Horizontal Scaling
- Load Balancing
- Database Replication
- Read Replicas
- Search Indexing
- Incremental Sync
- Webhooks
- Health Checks
- Distributed Logging
- Metrics & Monitoring

27. Suggested Tech Stack

Layer	Technology
Frontend	Next.js, React, TypeScript, Tailwind CSS, TanStack Query
Mobile	React Native or Flutter

Layer	Technology
Backend	NestJS (preferred) or Fastify
Database	PostgreSQL
Cache	Redis
Search	OpenSearch / Elasticsearch
Vector Search	pgvector or Qdrant
Queue	BullMQ, RabbitMQ, or Kafka
Object Storage	AWS S3 / Cloudflare R2
Authentication	OAuth2, JWT, Passport
AI	OpenAI, Gemini, or local embedding models
Monitoring	Prometheus + Grafana
Logging	Loki / ELK Stack
CI/CD	GitHub Actions
Containerization	Docker
Orchestration	Kubernetes (production)

28. Future Roadmap

Phase 1

- User authentication
- Job aggregation
- Search engine
- Resume parsing
- Basic recommendations

Phase 2

- AI semantic search
- Resume matching
- Company intelligence
- Salary insights
- Application tracking

Phase 3

- GitHub & portfolio intelligence
- AI career coach
- One-click apply

- Employer portal
- Mobile apps

Phase 4

- Global multi-language support
- Advanced AI recommendations
- Career analytics
- Learning integrations
- AI recruiter assistant
- AI-powered hiring workflows

Ultimate Goal

The platform should function as a complete **AI Career Operating System** rather than a traditional job board. It should intelligently aggregate opportunities, understand users through resumes, profiles, and developer portfolios, personalize search with AI, simplify applications, provide career guidance, and offer employers powerful AI-assisted recruitment tools—all within a scalable, microservice-friendly architecture suitable for enterprise deployment.

Pasted text.txt

Document

this is the current schema prisma

I don't actually have the Prisma schema yet. The attachment wasn't included in the conversation, so I can't review it.

Please upload one of these:

- `schema.prisma` (preferred)
- Or paste the schema directly into the chat
- Or upload your backend project as a ZIP if the schema depends on multiple files

Once I have it, I'll review it as if I were the lead backend architect and cover:

1. Database Architecture

- Normalize vs denormalize
- Missing entities
- Entity relationships
- Scalability

- Multi-tenancy readiness
- Soft delete strategy
- Audit fields
- Versioning

2. Job Platform Models

I'll compare your schema against the features we discussed and identify missing models such as:

- Job
- Company
- JobSource
- JobAggregation
- JobSearchHistory
- SavedSearch
- JobAlert
- Resume
- ResumeVersion
- ResumeEmbedding
- UserEmbedding
- Portfolio
- GitHubProfile
- GitHubRepository
- Skill
- UserSkill
- JobSkill
- CompanyReview
- SalaryInsight
- JobRecommendation
- AIRecommendation
- SearchAnalytics
- Employer
- Recruiter
- Interview
- Offer
- ApplicationTimeline
- CandidateProfile

- CareerGoal
- Notification
- Device
- SearchCache
- VectorEmbedding
- AIConversation
- SkillGap
- LearningRecommendation
- CareerRoadmap
- UserPreference
- LanguagePreference
- SearchSuggestion
- TrendingSearch
- JobView
- CompanyFollow
- RecruiterFollow
- SavedCompany
- EmployerAnalytics
- FraudDetection
- ATSIntegration
- JobImportLog
- APIKey
- Webhook
- ActivityLog

3. Search Optimization

I'll redesign it for:

- PostgreSQL
- Prisma
- Elasticsearch/OpenSearch
- Redis
- pgvector

including:

- proper indexes

- composite indexes
 - full-text search
 - vector search
 - caching strategy
 - pagination strategy
 - query optimization
-

4. AI Readiness

I'll make the schema ready for:

- Resume embeddings
 - Job embeddings
 - Semantic search
 - RAG
 - AI recommendations
 - GitHub analysis
 - Portfolio analysis
 - AI interview preparation
 - AI career coach
-

5. Enterprise Readiness

I'll check:

- RBAC
 - Permissions
 - Organizations
 - Teams
 - Audit logs
 - Event sourcing (if useful)
 - Background jobs
 - Notifications
 - Search indexing
 - Analytics
-

6. API Design

I'll also design a production-ready REST API (or GraphQL if you prefer), including endpoints such as:

- `/jobs/search`
 - `/jobs/recommendations`
 - `/jobs/trending`
 - `/jobs/similar`
 - `/resume/analyze`
 - `/resume/match`
 - `/profile/recommendations`
 - `/company/:id`
 - `/salary/insights`
 - `/portfolio/analyze`
 - `/github/analyze`
 - `/search/suggestions`
 - `/alerts`
 - `/applications`
 - `/analytics`
-

7. Frontend Architecture

I'll map the schema to a modern frontend architecture:

- Search page
 - AI search experience
 - Job details
 - Company pages
 - Resume dashboard
 - Portfolio dashboard
 - Career insights
 - Salary insights
 - Recommendation feed
 - Employer dashboard
 - Admin dashboard
-

8. Final Deliverable

I'll provide:

-  Improved Prisma schema
-  ER diagram

- ☒ Database architecture
- ☒ API specification
- ☒ Backend folder structure (NestJS)
- ☒ Frontend folder structure (Next.js)
- ☒ Elasticsearch/OpenSearch index mapping
- ☒ Redis caching strategy
- ☒ AI pipeline architecture
- ☒ Search architecture
- ☒ Scalable microservice roadmap
- ☒ Production deployment architecture

The goal will be to evolve your current schema into an enterprise-grade foundation suitable for an AI-powered career platform rather than just a conventional job portal.

now give me the full picture with all the things all and every details an noob intern can understand properly step by step way what i trying to achieve

This is exactly how I would explain the project to a new intern joining the team. It's written as a product + engineering specification so everyone (frontend, backend, AI, QA, DevOps) understands the end goal.

AI Career Platform

Complete Vision & Engineering Specification

1. What Are We Building?

We are **NOT** building another job portal like Naukri.

We are **NOT** building another Indeed clone.

We are building an **AI Career Intelligence Platform**.

Think of it as:

Google Search + LinkedIn Jobs + Glassdoor + GitHub + ChatGPT + ATS + AI Career Coach

inside one platform.

The goal is to become the smartest place where a person can:

- Search jobs
- Find better jobs
- Get AI recommendations
- Improve resume

- Track applications
- Analyze skills
- Learn missing skills
- Prepare for interviews
- Grow career

Everything should revolve around helping the candidate.

2. Problem Statement

Today's job search is broken.

A user usually visits

- LinkedIn
- Indeed
- Naukri
- Wellfound
- Glassdoor
- Company career pages

and repeats the same search.

They upload resumes again and again.

They never know

- Which job fits them best.
- Why they are rejected.
- Which skills are missing.
- Which company is best.
- Whether salary is fair.
- Which jobs are scams.

We solve all these problems.

3. Our Solution

Instead of searching manually,

our platform searches everywhere.

Instead of showing every job,

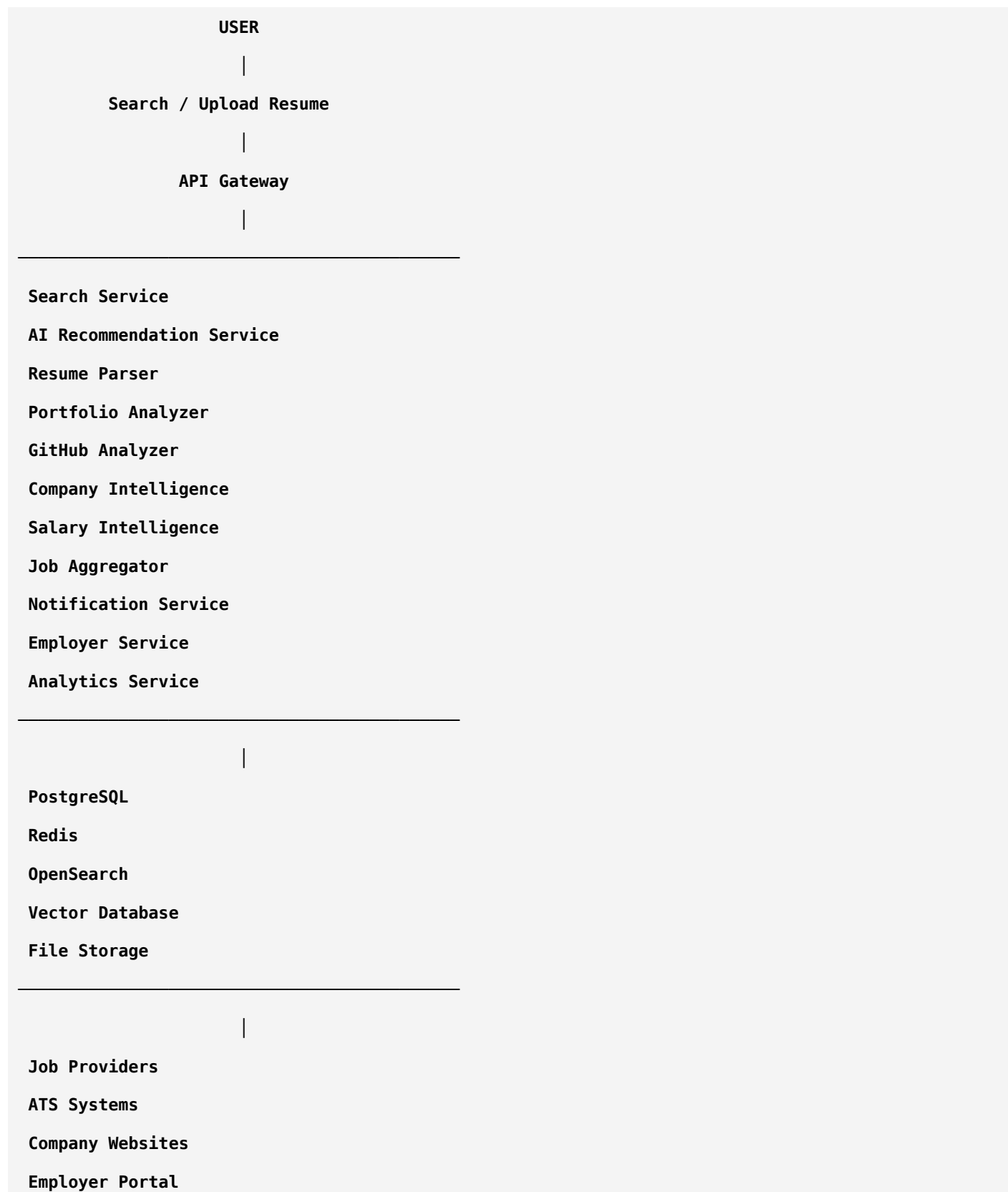
AI ranks jobs.

Instead of asking users to guess,

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

AI explains why jobs match.
 Instead of simply applying,
 AI improves their resume first.

4. High-Level Architecture



5. Complete User Journey

Imagine a new user named Rahul.

He visits our website.

Step 1

Creates account.

Step 2

Uploads resume.

Step 3

AI reads resume.

Extracts

- Skills
 - Experience
 - Education
 - Projects
 - Certifications
-

Step 4

Creates profile automatically.

Step 5

User connects GitHub.

Step 6

AI analyzes repositories.

Finds

- React
- Next.js
- Docker
- AWS

Step 7

AI creates skill profile.

Step 8

User searches

Frontend jobs

Step 9

Search service searches multiple providers.

Google Jobs

Jooble

Adzuna

RemoteOK

Greenhouse

Lever

Employer Jobs

Step 10

Results are merged.

Duplicates removed.

Step 11

AI ranks results.

Instead of

500 random jobs

User gets

Best Match
95%
React
Next.js
Remote
₹25L
Company Rating 4.8

Step 12

User opens job.

AI explains

You match because
✓ **React**
✓ **TypeScript**
✓ **REST API**
Missing
Docker
AWS

Step 13

AI suggests

Improve resume before applying.

Step 14

AI creates optimized resume.

Step 15

One-click apply.

Step 16

Track application.

Entire process is AI assisted.

6. Platform Modules

Everything is divided into modules.

Authentication

Handles

- Login
 - Register
 - Google Login
 - GitHub Login
 - LinkedIn Login
 - Password Reset
 - MFA
-

User Module

Stores

Profile

Resume

Experience

Education

Skills

Preferences

Career Goals

Languages

Portfolio

GitHub

Resume Module

Upload resume.

AI extracts
Skills
Projects
Education
Experience
Certificates
ATS Score
Resume Score
Missing Skills

Portfolio Module

Analyze
Website
Behance
Dribbble
GitHub
Figma
CodePen

GitHub Module

Read repositories.
Extract
Languages
Frameworks
Libraries
Architecture
Commit Activity
Project Quality
Open Source Score

Search Module

Search everywhere.

Supports
Keyword Search
AI Search
Voice Search
Semantic Search
Boolean Search
Location Search
Salary Search
Company Search
Skill Search
Natural Language Search

Job Aggregator

Collects jobs from
Employer Portal
Greenhouse
Lever
Adzuna
Jooble
Remotive
RemoteOK
Company Career Pages

Recommendation Engine

Personalized jobs based on
Resume
Skills
Experience
GitHub
Portfolio
Search History
Applied Jobs
Saved Jobs

AI Engine

Responsible for
Resume Parsing
Job Ranking
Semantic Search
Match Score
Cover Letter
Career Advice
Interview Questions
Learning Suggestions

Salary Engine

Provides
Average Salary
City Salary
Company Salary
Growth
Tax
PPP
Currency Conversion

Company Intelligence

Shows
Ratings
Benefits
Reviews
Funding
Tech Stack
Growth
Hiring Trend

Employer Module

Allows companies to

Post Jobs

View Candidates

Rank Candidates

Track Applicants

Interview Scheduling

Admin Module

Manage

Users

Companies

Jobs

Reports

Analytics

API Health

Logs

7. Search Flow

User Types

React Developer Remote

↓

Frontend

↓

API

↓

Search Service

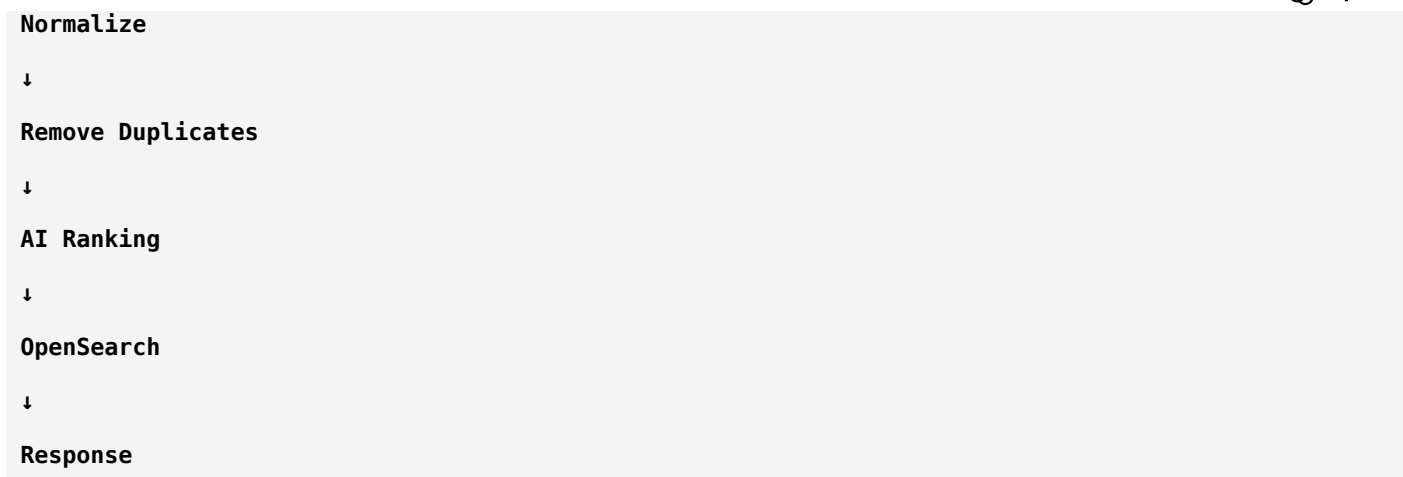
↓

Parallel Search

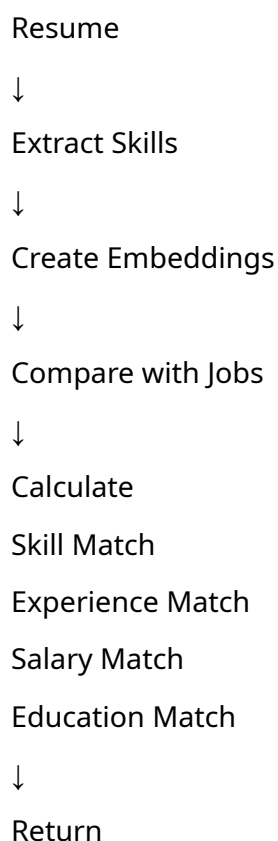
↓

Job APIs

↓



8. AI Resume Matching



92%

9. AI Search

User can type

```

Remote frontend jobs above 20L
Jobs similar to Amazon
Startups hiring React developers
Visa sponsorship backend jobs
  
```

Instead of keyword search,
AI understands meaning.

10. AI Career Coach

Acts like ChatGPT.

User asks

How do I become a Senior React Developer?

AI responds with

Learning Path

Skills

Projects

Courses

Jobs

Salary

Timeline

11. GitHub Intelligence

Reads repositories.

Finds

Languages

Frameworks

Libraries

Architecture

Testing

DevOps

Cloud

Creates

Developer Profile

12. Search Ranking

Every job gets scores.

Resume Match

Profile Match

GitHub Match
Portfolio Match
Salary Match
Location Match
Company Match
Freshness
Popularity
Overall AI Score
Highest score appears first.

13. Notifications

Notify user when
Dream company hiring
Salary increases
Resume needs update
Application status changes
Interview reminder
New matching jobs

14. Analytics

User Dashboard
Applications
Interviews
Resume Score
Profile Strength
Average Match
Top Skills
Missing Skills

Employer Dashboard
Applications
Hiring Funnel
Candidate Ranking
Job Performance

Admin Dashboard

Total Jobs

Searches

Revenue

API Health

Users

Employers

15. Database Overview

Main entities include:

- User
- Resume
- ResumeVersion
- Portfolio
- GitHubProfile
- GitHubRepository
- Skill
- UserSkill
- Education
- Experience
- Certification
- Job
- Company
- Employer
- Recruiter
- JobSource
- JobSkill
- JobLocation
- Application
- Interview
- Offer
- SavedJob
- SavedSearch
- JobAlert

- Notification
 - SalaryInsight
 - CompanyReview
 - JobRecommendation
 - SearchHistory
 - SearchAnalytics
 - AIConversation
 - CareerGoal
 - SkillGap
 - LearningRecommendation
 - ActivityLog
-

16. Technologies

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- TanStack Query
- Zustand/Redux
- React Hook Form

Backend

- NestJS
- Prisma ORM
- PostgreSQL
- Redis
- OpenSearch
- pgvector (or Qdrant)
- BullMQ

AI

- LLM for reasoning
- Embedding model
- Resume parser

- OCR for scanned resumes
- RAG for career assistance

Infrastructure

- Docker
 - Kubernetes
 - GitHub Actions
 - Prometheus
 - Grafana
 - S3-compatible storage
 - CDN
-

17. Development Phases

Phase 1 — Foundation

- Authentication
- User profiles
- Employer portal
- Job CRUD
- Resume upload
- Search
- Basic filters

Phase 2 — Intelligent Search

- Job aggregation
- OpenSearch
- Search suggestions
- Saved searches
- Alerts
- Company pages

Phase 3 — AI Personalization

- Resume parsing
- Match scoring
- Semantic search
- Recommendations

- Skill-gap analysis

Phase 4 — Career Intelligence

- Salary insights
- Company intelligence
- GitHub analysis
- Portfolio analysis
- AI career coach
- Learning roadmap

Phase 5 — Enterprise Scale

- Multi-language support
- Mobile apps
- One-click apply
- Employer AI
- Analytics
- Global expansion

Final Vision

The end goal is to build an **AI-powered Career Operating System**, not just a job board.

A traditional job portal only lists jobs. This platform will **understand the user, understand the market, and use AI to connect the right opportunity with the right person**. It will aggregate jobs from multiple legitimate sources, provide semantic search and intelligent ranking, analyze resumes and developer portfolios, recommend career improvements, automate parts of the application process, and support employers with AI-assisted recruitment.

In simple terms:

Google finds information. LinkedIn connects professionals. Glassdoor provides company insights. GitHub showcases developer work. ChatGPT provides intelligence.

This platform combines all of those capabilities into a single, AI-first career ecosystem.

user can upload resume or user can check by the search and this additional user information will be sent along with the user search that we have stored

Exactly. This is actually one of the core ideas that will make your platform much smarter than a traditional job board.

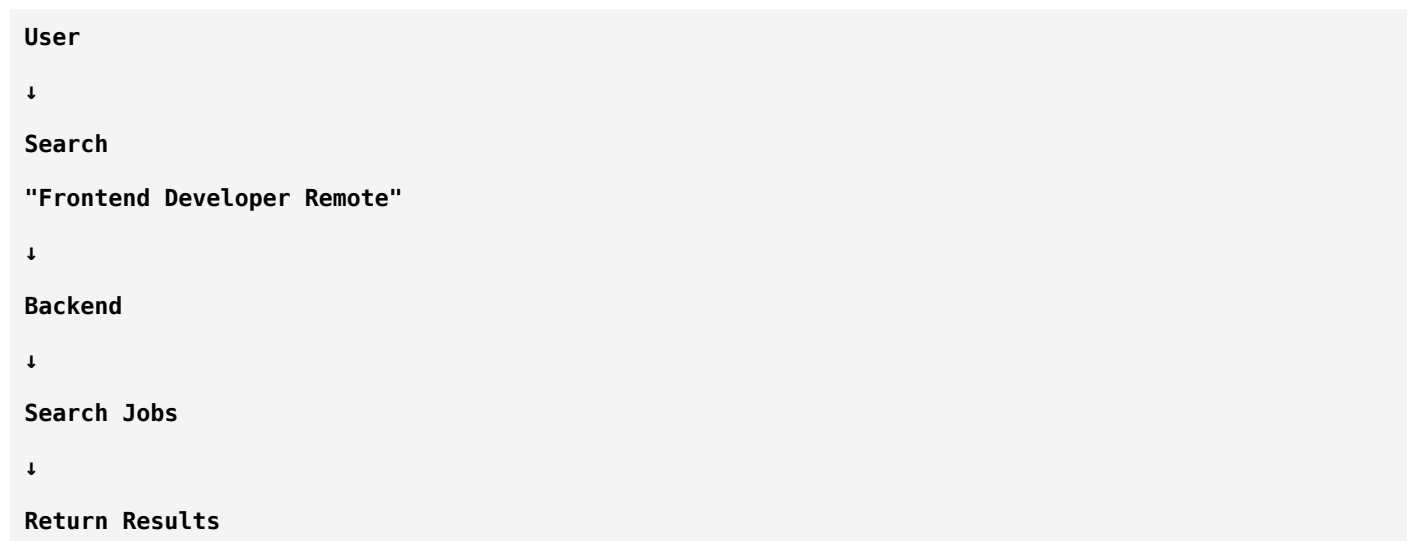
I would define it like this.

User Search Flow

There are **three ways** a user can search for jobs.

Flow 1 — Normal Search (Guest or New User)

The user hasn't uploaded a resume yet.



The backend only uses the search query and selected filters.

Request

```

JSON
{
  "query": "Frontend Developer",
  "location": "Bangalore",
  "remote": true,
  "salaryMin": 1200000,
  "experience": 2
}
  
```

Flow 2 — Profile-Based Search

The user has already completed their profile.

The system already knows:

- Skills
- Experience
- Education
- Preferred Location
- Preferred Salary
- Preferred Companies
- Preferred Work Type

- Languages
- Certifications

When the user searches

React Developer

the backend automatically enriches the search.

Instead of only searching

React Developer

the backend internally searches

Query:
React Developer

User Profile

Experience:
2 Years

Skills

React
Next.js
TypeScript
Node.js

Preferred Salary

15 LPA

Preferred Location

Bangalore

Remote

Yes

Preferred Companies

Product Based

Career Goal

Frontend Engineer

The search becomes much more personalized.

Flow 3 — Resume-Based Search

User uploads

resume.pdf

↓

Resume Parser

↓

Extract

```
React
Next.js
Docker
AWS
GraphQL
TypeScript
5 Years Experience
Product Companies
Education
Projects
```

↓

Store

```
Resume Profile
```

↓

Whenever user searches

```
Frontend Jobs
```

Backend sends

```
JSON
{
  "query": "Frontend Jobs",
  "resume": {
    "skills": [
      "React",
      "Next.js",
      "Docker",
      "AWS",
      "GraphQL"
    ],
    "experience": 5,
    "education": "B.Tech",
    "preferredLocation": "Remote",
    "expectedSalary": 2200000
  }
}
```

AI ranking becomes much more accurate.

Complete Search Pipeline

```
User Search
↓
API Gateway
↓
Authentication
↓
Load User Profile
↓
Load Resume
↓
Load Preferences
↓
Load Search History
↓
Load Saved Jobs
↓
Load Applied Jobs
↓
Load Portfolio
↓
Load GitHub
↓
Create Search Context
↓
Job Aggregator
↓
Search Providers
↓
Merge Jobs
↓
Remove Duplicates
↓
AI Match Engine
↓
Ranking Engine
↓
```

Response

Search Context

This is probably the most important object in the entire platform.

Instead of searching only with a keyword,
we search with the **complete user context**.

Example

```
TypeScript
SearchContext {
  query
  filters
  skills
  experience
  education
  projects
  technologies
  preferredLocations
  preferredSalary
  preferredCompanies
  remotePreference
  careerGoals
  languages
  resumeSkills
  portfolioSkills
  githubSkills
  savedJobs
  appliedJobs
  ignoredJobs
  recentSearches
  userPreferences
}
```

This object is passed to the Recommendation Engine.

Search API

Instead of


```
http
GET /jobs?query=React
```

We'll have

```
http
POST /api/jobs/search
```

Body

```
JSON
{
  "query": "React Developer",
  "filters": {
    "location": "Bangalore",
    "remote": true,
    "salaryMin": 1500000
  },
  "includeUserContext": true,
  "includeResume": true,
  "includePortfolio": true,
  "includeGithub": true
}
```

Notice something.

The frontend **doesn't send** the user's entire resume or profile every time.

It only sends

```
JSON
{
  "includeUserContext": true
}
```

The backend already knows the authenticated user.

It loads everything from the database.

Backend Search Context Builder

Search Request

↓

Get User

↓

Load Profile

↓

Load Resume

↓

Load Portfolio

↓

Load GitHub

↓

Load Skills

↓

Load Saved Jobs

↓

Load Search History

↓

Load Preferences

↓

Create Context

↓

Search Engine

This keeps the frontend lightweight and secure.

AI Ranking

Suppose the search returns

400 Jobs

Now AI scores every job.

Score calculation

Search Query

20%

Resume Match

25%

Profile Match

20%

GitHub Match

10%

Portfolio Match

10%

Location Match

5%
Salary Match
5%
Freshness
3%
Popularity
2%

Total

AI Score
96%

Sort descending.

Why This Is Better Than LinkedIn

LinkedIn mostly ranks jobs based on:

- Keywords
- Connections
- Basic profile information

Your platform can rank using **far richer context**, including:

- Search Query
- User Profile
- Resume Analysis
- GitHub Analysis
- Portfolio Analysis
- Previous Searches
- Saved Jobs
- Applied Jobs
- Ignored Jobs
- Career Goals
- Learning Progress
- Skill Gap Analysis
- Preferred Companies
- Preferred Technologies
- Preferred Work Style

- Salary Expectations
- Location Preferences

This transforms the experience from **"show me jobs that match my keywords"** to **"show me the jobs I'm most likely to succeed in and want to pursue."**

Recommendation

I would also introduce a dedicated **Search Context Service**. Rather than every service independently querying the database for user data, this service assembles a single **SearchContext** object from the user's profile, resume, preferences, GitHub, portfolio, and activity. The Search Service, Recommendation Engine, and AI Matching Engine all consume this unified context, making the architecture cleaner, easier to maintain, and more scalable.

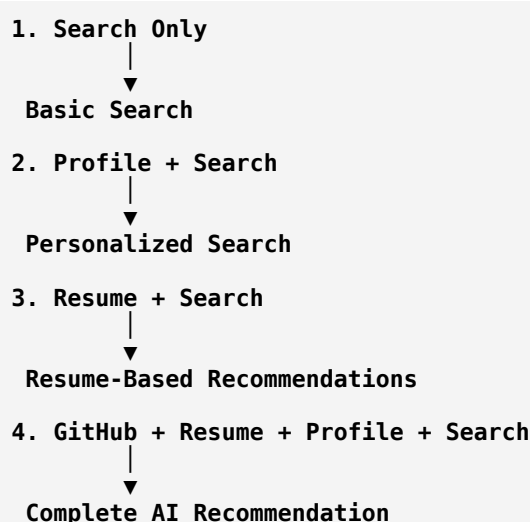
user can sent the github link of there profile then by that also this system gives the recommendation and make sure the empty githubs are taken care

Yes, and I think this is one of the strongest features you can build—especially for software engineers. However, you should design it so that **GitHub is an optional signal**, not a requirement. The recommendation engine should gracefully fall back to other data sources when GitHub is missing or not useful.

GitHub-Based Recommendation System

User Journey

The user can improve job recommendations in four ways:



Every additional source of information increases the confidence of the recommendation.

GitHub Integration

The user can connect GitHub by:

- Entering their GitHub profile URL
- OAuth login with GitHub (recommended)
- Username only (optional)

Example:

```
https://github.com/johndoe
```

or

```
johndoe
```

What the System Should Analyze

Profile Information

- Name
- Bio
- Company
- Location
- Followers
- Following
- Public Repositories
- Organizations
- Website
- Email (if public)
- Profile Readme

Repository Analysis

For each repository:

- Repository Name
- Description
- Primary Language
- Frameworks
- Libraries
- Topics
- Stars

- Forks
 - Watchers
 - Last Updated
 - README
 - License
 - Contributors
 - Open Issues
 - Pull Requests
-

Technical Skill Extraction

Instead of trusting repository names, inspect the codebase (where possible) and metadata to identify:

Languages:

- JavaScript
- TypeScript
- Python
- Java
- Go
- Rust
- C#
- PHP

Frameworks:

- React
- Next.js
- Angular
- Vue
- Express
- NestJS
- Django
- Flask
- Spring Boot

Databases:

- PostgreSQL
- MongoDB

- MySQL
- Redis

Cloud:

- AWS
- Azure
- GCP

DevOps:

- Docker
- Kubernetes
- GitHub Actions
- Terraform

Testing:

- Jest
- Cypress
- Playwright
- Vitest

AI Developer Profile

From GitHub, generate a structured profile:

Developer Profile

Primary Role:
Frontend Engineer

Secondary Role:
Full Stack Developer

Experience Level:
Intermediate

Languages:
TypeScript
JavaScript

Frameworks:
React
Next.js
Node.js

Databases:
PostgreSQL

Cloud:
AWS

DevOps:
Docker

Testing:
Jest

Open Source:
Moderate

Architecture Knowledge:
Good

Estimated Skill Confidence:
91%

This profile feeds directly into the recommendation engine.

GitHub Recommendation Flow

```

User connects GitHub
↓
Fetch profile
↓
Fetch repositories
↓
Analyze repositories
↓
Extract technologies
↓
Build GitHub Skill Profile
↓
Store in database
↓
Merge with Resume
↓
Merge with User Profile
↓
Create Search Context
↓
Search Jobs
↓
AI Match
↓
Return Recommendations

```

Handling Empty or Weak GitHub Profiles

This is very important.

Many users will have:

- No repositories
- Only forked repositories
- One "Hello World" repository
- Inactive accounts
- Private repositories only

The platform should **never penalize** users for this.

Instead, calculate a **GitHub Confidence Score**.

Scenario 1 — No GitHub Connected

GitHub Connected

No

Result:

Recommendation Source

Resume

+

Profile

+

Search Query

No warning is shown.

Scenario 2 — Empty GitHub

Repositories

0

Recommendation:

GitHub Score

Unavailable

Reason

No public repositories found.

Recommendations continue using your resume and profile.

No reduction in recommendation quality.

Scenario 3 — Private Repositories Only

Repositories

0 Public

Private Repositories

Unknown

Recommendation:

GitHub Analysis

Limited

Reason

Your repositories are private.

Recommendations are based on your resume and profile instead.

Scenario 4 — Mostly Forked Repositories

Repositories

15

Forked

14

Result:

GitHub Confidence

Low

Only one original project detected.

Original repositories provide better skill analysis.

Still continue with recommendations.

Scenario 5 — Inactive GitHub

Last Commit

3 years ago

Recommendation:

GitHub Activity

Low

We'll prioritize your resume and profile for recommendations.

Scenario 6 — Excellent GitHub

Repositories

45

Stars

250

Commits

3200

Recent Activity

Yes

GitHub becomes a strong recommendation signal.

Confidence-Based Recommendation Engine

Every information source contributes differently.

Search Query

100%

Resume

95%

User Profile

90%

GitHub

0–95%

Portfolio

0–90%

Learning History

80%

Saved Jobs

70%

Applied Jobs

65%

Search History

60%

The engine adjusts weights dynamically.

Example:

GitHub Empty

```
↓
GitHub Weight = 0%
Resume Weight = Increased
Profile Weight = Increased
```

The user still gets excellent recommendations.

Search Context

Every search builds a unified context:

```
TypeScript
SearchContext {
  query
  filters
  profile
  resume
  github
  portfolio
  preferences
  careerGoals
  searchHistory
  savedJobs
  appliedJobs
  skills
  experience
  education
  languages
}
```

Every module can contribute data.

If GitHub isn't available:

```
TypeScript
github = null
```

Everything still works.

AI Explanation

Instead of simply showing:

92% Match

Explain the reasoning:

Why this job matches

- ✓ React expertise identified in your GitHub projects
- ✓ TypeScript experience from your resume
- ✓ Next.js projects detected
- ✓ Preferred remote work
- ✓ Salary expectation matches

Missing Skills

Docker

AWS

CI/CD

Or, if GitHub wasn't used:

Why this job matches

- ✓ React experience from your resume
- ✓ Three years of frontend development
- ✓ Preferred location

GitHub analysis wasn't available because no public repositories were found.

Anti-Abuse & Validation

To keep recommendations reliable:

- Ignore repositories with no meaningful content.
- Reduce confidence for repositories containing only generated boilerplate.
- Ignore repositories with no commits beyond the initial scaffold.
- Detect duplicate/template repositories.
- Identify portfolio-only repositories versus real development work.
- Analyze contribution history over time instead of repository count alone.
- Re-analyze GitHub periodically or when the user requests a refresh.

Engineering Principle

Treat every data source as an **optional enrichment layer**.

Search Query

+

User Profile

```

+
Resume
+
GitHub
+
Portfolio
+
User Activity
+
Preferences
+
Career Goals
=
Unified Search Context

```

The recommendation engine should never assume all inputs are present. It should produce high-quality recommendations from whatever information is available, while transparently explaining which sources influenced the results. This makes the system robust, user-friendly, and applicable to both experienced engineers with rich GitHub profiles and candidates who have never used GitHub at all.

list all the feature our job search will have compared what other offers and

If your goal is to compete with LinkedIn, Google Jobs, Indeed, Glassdoor, Wellfound, etc., then your search shouldn't just be a **search bar**—it should be an **AI-powered Job Discovery Engine**.

Below is a comprehensive feature list, organized by category, including what competitors typically offer and where your platform can differentiate.

1. Search Methods

Standard Search

- Keyword Search
- Job Title Search
- Company Search
- Skill Search
- Technology Search
- Industry Search
- Location Search
- Recruiter Search
- Organization Search

AI Search

- Natural Language Search
- Semantic Search
- Intent-Based Search
- Conversational Search

- AI Query Understanding
- AI Query Expansion
- AI Query Correction
- AI-Powered Job Discovery
- AI Explain Search Results

Developer Search

- Search by Programming Language
 - Search by Framework
 - Search by Tech Stack
 - Search by Open Source Technologies
 - Search by Cloud Platform
 - Search by DevOps Stack
-

2. Search Filters

Location

- Country
- State
- City
- Area
- Radius Search
- Remote
- Hybrid
- On-site
- Anywhere
- Time Zone

Employment Type

- Full Time
- Part Time
- Internship
- Contract
- Freelance

- Temporary
- Apprenticeship
- Volunteer

Experience

- Fresher
- Entry Level
- Junior
- Mid-Level
- Senior
- Lead
- Architect
- Manager
- Director
- Executive

Salary

- Minimum Salary
- Maximum Salary
- Currency
- Monthly
- Annual
- Hourly
- Total Compensation
- Equity
- Bonus

Company

- Startup
- Scale-up
- Unicorn
- Enterprise
- Government

- NGO
- Fortune 500
- Verified Employer

Work Preference

- Remote
- Hybrid
- Flexible Hours
- Four-Day Week
- Night Shift
- Rotational Shift

Benefits

- Health Insurance
- Dental
- Stock Options
- ESOP
- Learning Budget
- Gym
- Free Meals
- Paid Vacation
- Unlimited PTO
- Visa Sponsorship
- Relocation
- Work From Anywhere

3. AI-Powered Personalization (Your Biggest Differentiator)

Recommendations based on:

- User Profile
- Resume
- GitHub
- Portfolio
- Search History

- Viewed Jobs
 - Saved Jobs
 - Applied Jobs
 - Ignored Jobs
 - Preferred Companies
 - Preferred Salary
 - Preferred Technologies
 - Preferred Industries
 - Career Goals
 - Learning Progress
 - Previous Experience
 - Certifications
 - Languages
 - Skills
-

4. Resume Intelligence

- Resume Upload
 - Resume Parsing
 - ATS Score
 - Resume Score
 - Skill Extraction
 - Experience Extraction
 - Education Extraction
 - Project Extraction
 - Certification Extraction
 - Missing Skill Detection
 - Resume Improvement Suggestions
 - Resume Versioning
 - Resume Comparison
-

5. GitHub Intelligence (Developer-Focused)

- GitHub Profile Import
- Repository Analysis
- Technology Detection

- Programming Language Detection
 - Framework Detection
 - Cloud Detection
 - DevOps Detection
 - Testing Framework Detection
 - Contribution Analysis
 - Open Source Score
 - Repository Health
 - Code Quality Indicators
 - Activity Analysis
 - Developer Skill Matrix
 - GitHub Match Score
 - Graceful fallback for empty or private profiles
-

6. Portfolio Intelligence

Support:

- Personal Website
- GitHub
- GitLab
- Bitbucket
- Behance
- Dribbble
- Figma
- CodePen
- Medium
- Dev.to
- Stack Overflow
- Kaggle
- LeetCode
- HackerRank

Analysis:

- Portfolio Quality
- UI/UX
- Accessibility

- SEO
 - Performance
 - Mobile Responsiveness
 - Live Project Detection
 - Broken Links
 - Project Relevance
-

7. Search Suggestions

- Auto Complete
 - Trending Searches
 - Related Searches
 - AI Suggestions
 - Skill Suggestions
 - Company Suggestions
 - Technology Suggestions
 - Salary Suggestions
 - Location Suggestions
 - Career Suggestions
-

8. Smart Ranking

Every job receives scores such as:

- AI Match Score
 - Resume Match
 - GitHub Match
 - Portfolio Match
 - Skill Match
 - Experience Match
 - Salary Match
 - Company Match
 - Location Match
 - Career Goal Match
 - Freshness Score
 - Popularity Score
-

9. AI Explanations

Instead of:

87% Match

Show:

Why This Matches

✓ React

✓ TypeScript

✓ Next.js

✓ Node.js

Missing Skills

Docker

AWS

CI/CD

Estimated Interview Probability

81%

10. Company Intelligence

Every company page includes:

- Company Overview
- Industry
- Employee Count
- Funding
- Tech Stack
- Open Positions
- Reviews
- Salaries
- Benefits
- Hiring Trends
- Growth Trends
- Work Culture
- Interview Experience

11. Salary Intelligence

- Market Salary
- Company Salary

- City Salary
 - Country Salary
 - Cost of Living
 - Currency Conversion
 - PPP Adjustment
 - Tax Estimation
 - Net Salary
 - Salary Growth Prediction
 - Negotiation Suggestions
-

12. Application Features

- One-Click Apply
 - AI Cover Letter
 - Resume Selection
 - Multiple Resume Versions
 - Portfolio Attachment
 - Auto Form Fill
 - Application Tracking
 - Status Timeline
 - Recruiter Viewed
 - Interview Tracking
 - Offer Tracking
-

13. Job Alerts

- Instant Alerts
 - Daily Digest
 - Weekly Digest
 - AI Recommendations
 - Salary Alerts
 - Dream Company Alerts
 - Technology Alerts
 - Remote Job Alerts
-

14. Analytics

For Candidates:

- Applications
- Interviews
- Resume Views
- Recruiter Views
- Match Scores
- Skill Trends
- Missing Skills
- Success Rate

For Employers:

- Candidate Funnel
- Application Rate
- Job Performance
- Hiring Analytics

15. AI Career Coach

- Resume Review
- Career Advice
- Learning Roadmap
- Skill Gap Analysis
- Interview Preparation
- Salary Negotiation
- Career Switching Guidance
- AI Chat Assistant

16. Global Features

- Multi-language UI
- Job Translation
- Resume Translation
- Recruiter Message Translation
- Localized Search
- Global Salary Normalization
- Country-specific Filters

17. Mobile Experience

- Progressive Web App (PWA)
 - Native Mobile Apps
 - Voice Search
 - Camera Resume Upload
 - Offline Saved Jobs
 - Push Notifications
 - Biometric Login
 - Swipe Actions
-

18. Developer-Centric Features (Unique Selling Point)

- GitHub Integration
 - Portfolio Analysis
 - Tech Stack Detection
 - Open Source Recognition
 - Coding Profile Integration (LeetCode, HackerRank, Codeforces)
 - Developer Readiness Score
 - Project Quality Score
 - Recruiter Readiness Score
-

19. Search API Capabilities

Your search backend should support:

- Multi-source aggregation
- Parallel provider queries
- Result normalization
- Duplicate detection
- AI enrichment
- Personalized ranking
- Cursor pagination
- Faceted filtering
- Autocomplete
- Search analytics
- Caching

- Synonym support
- Typo tolerance

20. Enterprise Features

- Employer Dashboard
- ATS Integrations
- Candidate Ranking
- AI Resume Screening
- Recruiter Search
- Team Collaboration
- Role-Based Access Control
- Audit Logs
- API Access
- Webhooks

Competitive Comparison

Feature	LinkedIn	Google Jobs	Indeed	Glassdoor	Your Platform
Multi-source Job Aggregation	✗	✓	✗	✗	✓
AI Semantic Search	Limited	Limited	✗	✗	✓
Resume-Based Ranking	Basic	✗	Basic	✗	✓
GitHub-Based Recommendations	✗	✗	✗	✗	✓
Portfolio Analysis	✗	✗	✗	✗	✓
AI Match Explanations	✗	✗	✗	✗	✓
Skill Gap Detection	Limited	✗	✗	✗	✓
AI Career Coach	Limited	✗	✗	✗	✓
Global Salary Normalization	✗	✗	✗	Partial	✓
Personalized Search Context	Limited	✗	Limited	✗	✓
One-Click Apply (Supported Sources)	Partial	✗	Partial	✗	✓
GitHub/Portfolio Skill Extraction	✗	✗	✗	✗	✓
Transparent AI Match Scoring	✗	✗	✗	✗	✓
Multi-language AI Search	✗	Limited	✗	✗	✓

The Core Differentiator

The biggest advantage of your platform isn't any single feature—it's the **unified AI search context**.

Every search can intelligently combine:

- The user's search query
- Resume and extracted skills
- User profile and preferences
- GitHub repositories (if available)
- Portfolio analysis
- Search history
- Saved and applied jobs
- Career goals
- Market trends
- Salary expectations
- Company preferences

This creates a **personalized recommendation engine** rather than a keyword search engine. Instead of simply returning jobs that contain the word "React," the platform can prioritize opportunities that best match the user's overall profile, explain *why* they match, identify missing skills, and guide the user toward stronger career outcomes. That's a capability that most existing job platforms only provide in a limited way, if at all.

so as an developer any suggestions ??

Yes. If I were the lead backend architect on this project, there are quite a few things I would change before writing a single line of code. The features you've listed are excellent, but the **architecture** will determine whether the platform remains maintainable after 2–3 years.

Here are my recommendations.

1. Don't Think of It as a Job Search API

This is the biggest mindset shift.

Instead of

Job Search API

Think

Career Intelligence Engine

Searching is only one capability.

The platform should answer questions like

What jobs should I apply for?

Why didn't I get selected?

What skills should I learn?

What salary should I ask?

Which companies should I target?

2. Build Around "Search Context"

Instead of

```
search(query)
```

Everything should use

```
TypeScript  
SearchContext {  
  
  query  
  user  
  resume  
  github  
  portfolio  
  preferences  
  careerGoals  
  filters  
  searchHistory  
  savedJobs  
  appliedJobs  
  marketInsights  
}
```

Every AI feature will consume this object.

Never search only by keyword.

3. Separate Search and Recommendation

Many developers mix these.

Bad

```
Search API
```

```
↓
```

Returns recommendations

Good

Search Engine

↓

Returns candidate jobs

↓

Recommendation Engine

↓

Ranks jobs

↓

AI Engine

↓

Explains jobs

Three completely different services.

4. Build a Job Knowledge Graph

Instead of storing

React Developer

Store relationships.

React

↓

Frontend

↓

JavaScript

↓

TypeScript

↓

Redux

↓

Next.js

↓

REST

↓

GraphQL

Now searching

Next.js

can automatically include

React jobs.

Searching

Node

can include

NestJS

Express

Fastify

5. Every Job Needs AI Enrichment

Never trust provider data.

Instead

Provider Job

↓

Normalize

↓

AI Extract

↓

Store

Extract

- Skills
- Technologies
- Experience
- Soft Skills
- Responsibilities
- Benefits
- Salary
- Industry
- Team Size

- Seniority
- Visa
- Remote Policy

Now every job has consistent structure.

6. Don't Depend on Resume

Resume is optional.

Priority should be

```
Search Query
+
User Profile
+
Resume
+
GitHub
+
Portfolio
+
Activity
+
Preferences
+
Market Trends
```

Any one source can be missing.

Everything still works.

7. Introduce Confidence Scores

Every data source should have confidence.

Example

```
Resume
95%
GitHub
40%
Portfolio
```

85%

Profile

100%

Recommendation engine uses confidence.

8. Make Search Explainable

Instead of

94%

Explain

Matched because

React

TypeScript

Next.js

Remote

Salary

Missing

Docker

AWS

Trust increases dramatically.

9. Learn From User Behavior

After enough users,

track

Searches

Views

Clicks

Bookmarks

Applies

Rejects

Interviews

Offers

Eventually

AI learns

People who viewed this also applied here.

Netflix-style recommendations.

10. Use Event Driven Architecture

Instead of

Upload Resume

↓

Everything happens synchronously

Use

Resume Uploaded

↓

Queue

↓

Resume Parser

↓

Skill Extractor

↓

Embedding Generator

↓

Recommendation Update

↓

Notification

Much more scalable.

11. Everything Should Produce Embeddings

Generate embeddings for

Jobs

Resume

Skills

Companies

Projects

Portfolio

GitHub

Now semantic search becomes powerful.

12. AI Should Continuously Learn

Don't generate recommendations only when searching.

Whenever

Resume changes

↓

GitHub changes

↓

Company posts job

↓

Skill changes

↓

Salary changes

↓

Learning completed

Recalculate recommendations.

13. Create a Unified Skill Engine

Every module should use

Skill Service

instead of storing

React

in ten places.

Skill Engine knows

Aliases

Popularity

Demand

Salary

Related Skills

Learning Path

Companies

Jobs

14. Normalize Every Provider

Every provider returns different fields.

Never expose provider data directly.

Always convert to

```
TypeScript
NormalizedJob {
  id
  title
  company
  salary
  skills
  experience
  benefits
  location
  employmentType
  remote
  ...
}
```

Makes providers replaceable.

15. AI Search Should Support Intent

Examples

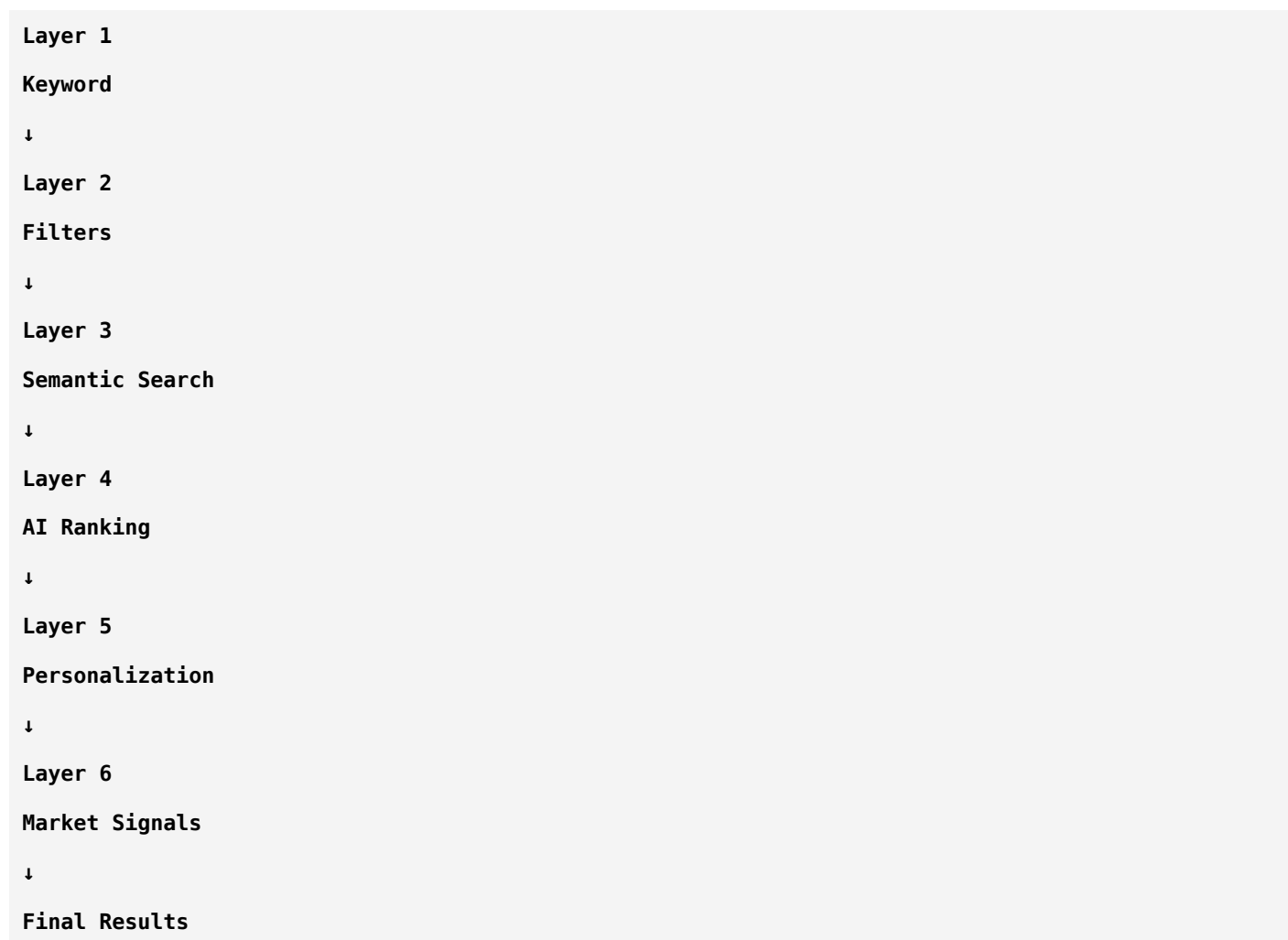
Instead of

React jobs

Users should type

```
I want remote frontend jobs paying above 25 LPA.
Companies similar to Google.
Startup jobs using AI.
Backend jobs with visa sponsorship.
Jobs matching my GitHub profile.
Find jobs better than my current one.
```

16. Search Should Be Layered

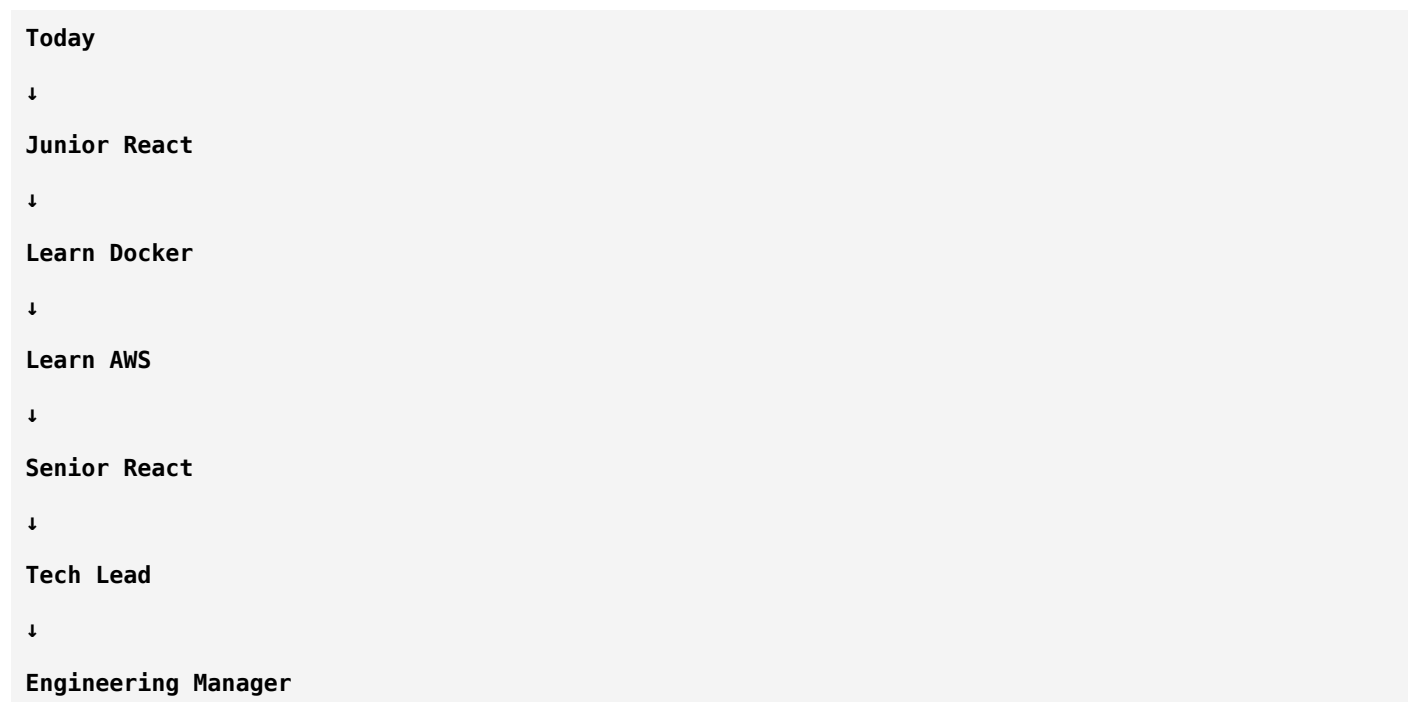


17. Don't Just Search Jobs

Search
Jobs
Companies
Skills
Technologies
Learning Paths
Certifications
Recruiters
Career Advice
Salary Reports
Market Trends
Events
Hackathons

18. AI Career Timeline

Imagine



AI predicts

Salary

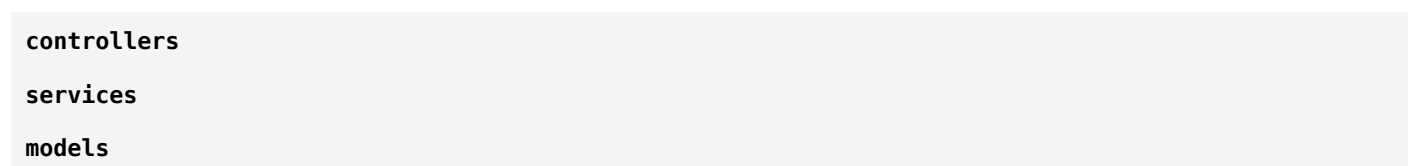
Timeline

Required Skills

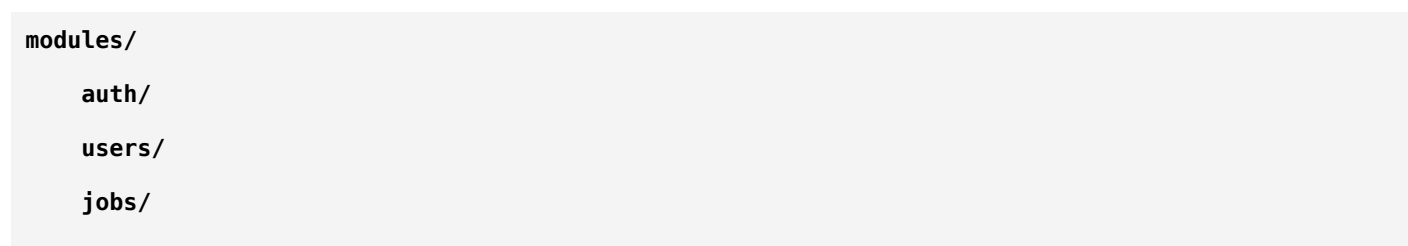
Recommended Jobs

19. Build Everything Around Features

Instead of



Structure by domain:



```
search/
recommendations/
ai/
resume/
github/
portfolio/
companies/
employers/
analytics/
notifications/
salary/
skills/
career/
admin/
```

This scales much better.

20. The Biggest Suggestion

If I had to add **one feature that almost no job platform has**, it would be this:

Career Digital Twin

Instead of storing

User

Store

Career Profile

The AI maintains a living model of the user's career.

It knows:

- Skills
- Technologies
- Learning history
- Projects
- Resume versions
- GitHub evolution
- Salary progression

- Interview history
- Companies applied to
- Companies rejected
- Career goals
- Strengths
- Weaknesses
- Preferred work style
- Market demand
- Future opportunities

Every recommendation, search result, and career suggestion is based on this continuously updated profile.

If This Were My Startup

I'd prioritize these features in order:

1. **Multi-source Job Aggregation** — become the single place to discover jobs.
2. **AI Search Context Engine** — personalize every search using user context.
3. **AI Match & Explainability** — don't just rank jobs; explain the ranking.
4. **Resume + GitHub + Portfolio Intelligence** — build the richest candidate profile possible.
5. **Career Intelligence Layer** — salary insights, skill gaps, career roadmap, and learning recommendations.
6. **Employer AI** — intelligent candidate ranking and hiring workflows.

One final recommendation: **design your platform as a pipeline, not as CRUD APIs.**

```

Collect Jobs
  ↓
Normalize Data
  ↓
AI Enrichment
  ↓
Index Search
  ↓
Personalize Results
  ↓
Recommend Jobs
  ↓
Explain Matches
  ↓
Track Outcomes
  ↓
Learn From Feedback
  ↓
Improve Future Recommendations
  
```

That mindset is what separates an AI-powered career platform from a traditional job board.

i want to make the backend to handle this in one api so tha i can apply this to the current system after creating this

Yes, and I actually think this is the **correct approach**.

I would **not** build 20 different APIs initially. Build **one intelligent search endpoint** that orchestrates everything behind the scenes. Later, you can split it into microservices without changing the frontend.

The Idea

The frontend only knows one endpoint:

```
http
POST /api/v1/jobs/search
```

The backend does everything.

Flow

```
User Search
↓
Authentication
↓
Create Search Context
↓
Search Aggregator
↓
Collect Jobs
↓
Normalize Jobs
↓
Remove Duplicates
↓
Enrich Jobs (AI)
↓
Apply Filters
↓
AI Ranking
↓
Recommendations
```

↓

Response

The frontend doesn't care whether jobs came from LinkedIn, Google Jobs, your own database, or another provider.

Request

JSON

```
{
  "query": "Frontend React Developer",
  "page": 1,
  "limit": 20,
  "filters": {
    "remote": true,
    "salaryMin": 1200000,
    "experience": 2,
    "employmentType": ["FULL_TIME"],
    "location": {
      "country": "India",
      "city": "Bangalore"
    }
  },
  "options": {
    "useProfile": true,
    "useResume": true,
    "useGithub": true,
    "usePortfolio": true,
    "aiRanking": true,
    "semanticSearch": true,
    "includeRecommendations": true
  }
}
```

Notice there is **no resume**, **no GitHub URL**, and **no profile** in the request.

The backend already knows the authenticated user.

Backend

SearchController

↓

SearchService

↓

ContextBuilder

↓

JobAggregator

↓

RankingEngine

↓

RecommendationEngine

↓

ResponseBuilder

Step 1

Search Controller

Receives

POST /jobs/search

Gets

User ID

Search Request

Step 2

Search Context Builder

Builds

```
TypeScript
SearchContext {
  user
  profile
  resume
  github
  portfolio
  preferences
  careerGoals
  searchHistory
  savedJobs
  appliedJobs
  filters
  query
}
```

This object is now passed everywhere.

Step 3

Job Aggregator

This service is responsible for talking to every provider.

```

Google Jobs
↓
LinkedIn
↓
Jooble
↓
Adzuna
↓
Remotive
↓
Employer Jobs
↓
Internal Jobs

```

Each provider has its own adapter.

```

GoogleAdapter
LinkedInAdapter
JoobleAdapter
AdzunaAdapter
EmployerAdapter

```

Each adapter returns

```

TypeScript
NormalizedJob

```

Not provider-specific data.

Step 4

Normalizer

Everything becomes

```

TypeScript
Job {
  id
  title
  company
}

```

```

salary
skills
description
benefits
experience
location
remote
postedAt
source
}

```

Now every provider looks identical.

Step 5

Duplicate Detection

Suppose

Google

returns

```
React Developer
```

Jooble

returns

```
React Developer
```

Employer

returns

```
React Developer
```

The system merges them into one canonical job while preserving all source URLs and metadata.

Step 6

AI Enrichment

For every job

Extract

- Skills

- Technologies
- Experience
- Seniority
- Salary
- Benefits
- Industry
- Department
- AI Summary
- Keywords
- Embeddings

Store them.

Step 7

Filtering

Apply

- Salary
 - Remote
 - Hybrid
 - Experience
 - Companies
 - Technologies
 - Visa
 - Benefits
-

Step 8

Recommendation Engine

Calculate

Resume Match

Profile Match

GitHub Match

Portfolio Match

Skill Match

Experience Match

Salary Match

Location Match
Career Goal Match
Popularity
Freshness

Generate

Final Score

Step 9

Sort

Sort by

AI Score
↓
Freshness
↓
Popularity

Step 10

Response Builder

Instead of

```
JSON
[
  {},
  {},
  {}
]
```

Return

```
JSON
{
  "jobs": [],
  "recommendations": [],
  "filters": {},
  "facets": {},
  "pagination": {},
  "searchInsights": {},
  "userInsights": {},
  "metadata": {}
}
```

Complete Backend Architecture



Make It Pipeline-Based

Instead of one massive service with 3,000 lines of code:

```

TypeScript
searchJobs()
  
```

Create a processing pipeline.

```

TypeScript
searchJobs()
  ↓
  buildContext()
  ↓
  collectJobs()
  ↓
  normalizeJobs()
  ↓
  
```

```

removeDuplicates()
↓
enrichJobs()
↓
filterJobs()
↓
rankJobs()
↓
generateRecommendations()
↓
buildResponse()

```

Each function should ideally do **one thing only**.

Suggested Folder Structure (NestJS)

```

modules/
  jobs/
    controllers/
      jobs.controller.ts
    services/
      search.service.ts
      context-builder.service.ts
      aggregator.service.ts
      ranking.service.ts
      recommendation.service.ts
      duplicate.service.ts
      enrichment.service.ts
      filter.service.ts
      response-builder.service.ts
    providers/
      linkedin.provider.ts
      google.provider.ts
      jooble.provider.ts
      adzuna.provider.ts
      employer.provider.ts
    interfaces/
    dto/

```

```
entities/  
utils/
```

Future-Proofing

If you later decide to move to microservices, nothing changes for the frontend.

Today:

```
Frontend  
↓  
POST /jobs/search  
↓  
Monolith
```

Later:

```
Frontend  
↓  
POST /jobs/search  
↓  
API Gateway  
↓  
Search Service  
↓  
Recommendation Service  
↓  
Aggregator Service  
↓  
AI Service  
↓  
Employer Service
```

The frontend still calls the same endpoint.

One Enhancement I'd Add

Instead of making `POST /jobs/search` just return jobs, make it a **search orchestration endpoint** that also returns everything the search page needs in a single response:

```
JSON  
{  
  "jobs": [...],
```



```

"recommendations": [...],
"relatedSearches": [...],
"suggestedSkills": [...],
"salaryInsights": {
  "average": 1800000,
  "range": "15L-22L"
},
"companiesHiring": [...],
"activeFilters": {...},
"facets": {
  "locations": [...],
  "companies": [...],
  "skills": [...],
  "employmentTypes": [...]
},
"searchSummary": {
  "totalResults": 1284,
  "personalized": true,
  "contextUsed": [
    "profile",
    "resume",
    "github"
  ]
},
"pagination": {
  "page": 1,
  "limit": 20,
  "hasNext": true
}
}

```

This allows your frontend to render the entire search experience from **one API call**, minimizing network requests and giving you the flexibility to evolve the backend implementation without changing the client contract.

give me an image how this will be looking on frontend ui

<https://github.com/speedyapply/JobSpy.git> this is another git hub repo i got

Yes, I know about **JobSpy**, and it's actually one of the better open-source projects to study. I wouldn't use it as-is, but I **would definitely use it as one of the data providers** for your platform.

What JobSpy Does Well

JobSpy is essentially a **job scraping/aggregation library**.

It can collect jobs from multiple providers like:

- LinkedIn
- Indeed
- Glassdoor
- Google Jobs (depending on availability and maintenance)
- ZipRecruiter
- Others (supported by the project/version)

It returns a normalized dataset, which is exactly what your **Job Aggregator Service** needs.

Where It Fits in Your Architecture

Instead of this:

```
Frontend
↓
Search API
↓
LinkedIn API
```

Think of this:

```
Frontend
↓
POST /api/jobs/search
↓
Search Orchestrator
↓
Job Aggregator
├── JobSpy Provider
├── Adzuna Provider
├── Jooble Provider
├── Employer Jobs
├── Internal Jobs
├── Future Providers
↓
Normalize
↓
Deduplicate
↓
AI Enrichment
↓
Recommendation Engine
↓
Response
```

JobSpy becomes **one provider**, not your entire backend.

I Would Wrap It

Don't call JobSpy directly from your controller.

Create an interface.

```
TypeScript
interface JobProvider {
  searchJobs(context: SearchContext): Promise<NormalizedJob[]>
}
```

Then

```
JobSpyProvider
implements
JobProvider
```

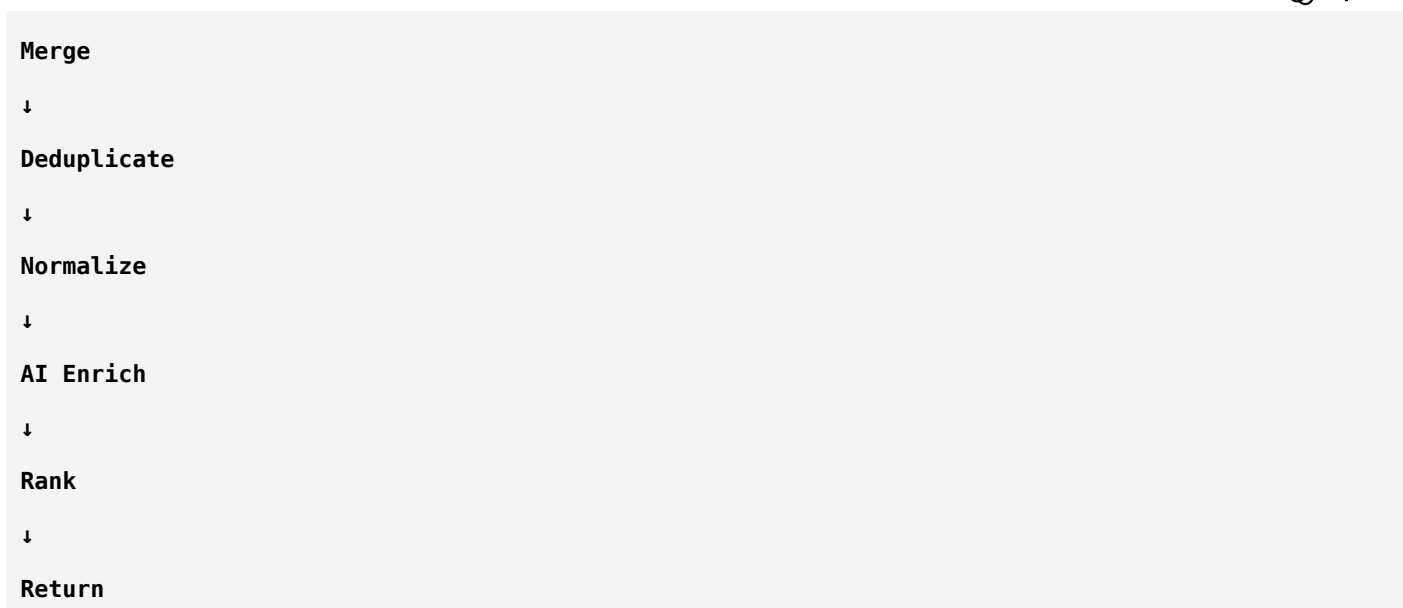
Later you can add

```
AdzunaProvider
JoobleProvider
LinkedInOfficialProvider
GoogleJobsProvider
EmployerProvider
InternalJobProvider
```

Your Search Service never knows which provider is being used.

Pipeline

```
Search
↓
Context Builder
↓
Provider Manager
↓
JobSpy
↓
Adzuna
↓
Jooble
↓
Employer Jobs
↓
```



What JobSpy Doesn't Do

This is important.

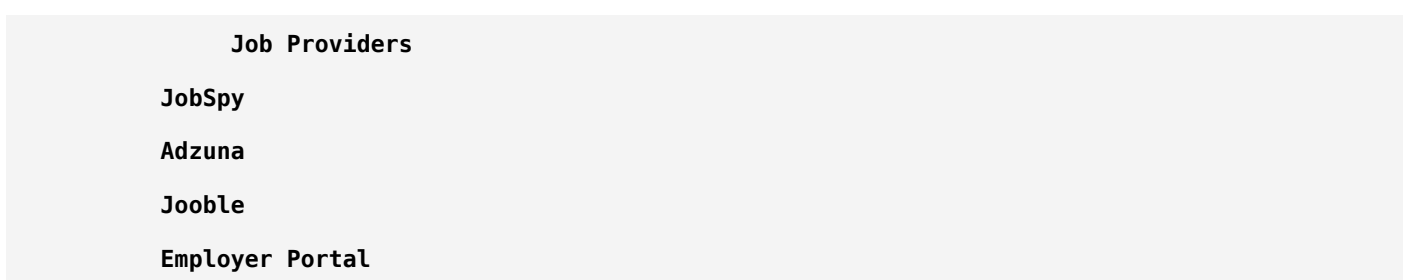
JobSpy only helps you **collect jobs**.

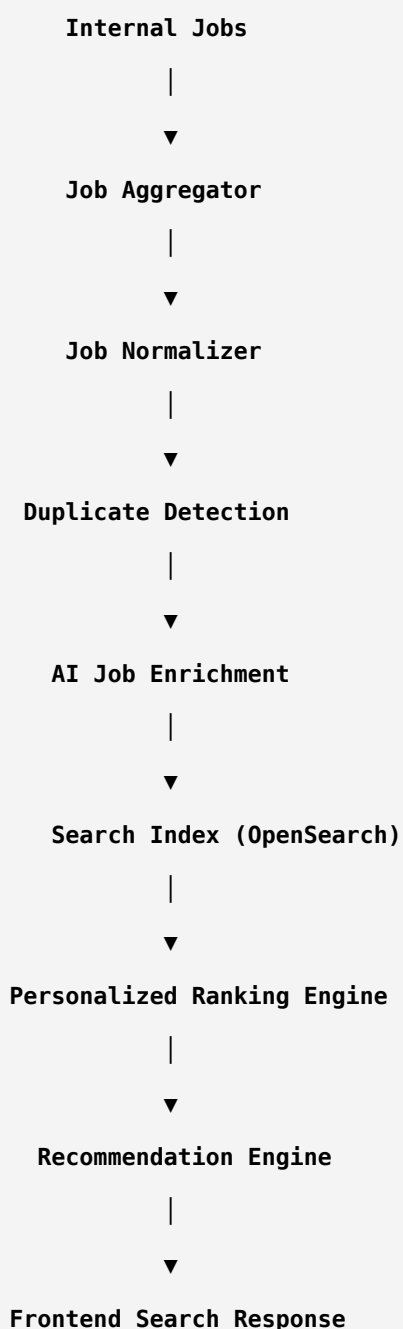
It does **NOT** provide:

- ✗ Resume Matching
- ✗ GitHub Analysis
- ✗ AI Ranking
- ✗ Portfolio Analysis
- ✗ Career Intelligence
- ✗ Salary Intelligence
- ✗ Recommendation Engine
- ✗ User Personalization
- ✗ Skill Gap Analysis
- ✗ Interview Probability
- ✗ AI Explanations

Those are your value proposition.

Your Architecture Should Be





Additional Repositories I'd Recommend Studying

These are useful for ideas, but I would avoid copying their architecture directly.

Job Aggregation

- JobSpy (multi-provider scraping)
- Career Ops (workflow inspiration)
- find-jobs (provider integrations)

Resume Parsing

- OpenResume
- Resume Parser projects using spaCy or transformer models

Search

- OpenSearch
- Elasticsearch examples
- Meilisearch (great developer experience)

AI

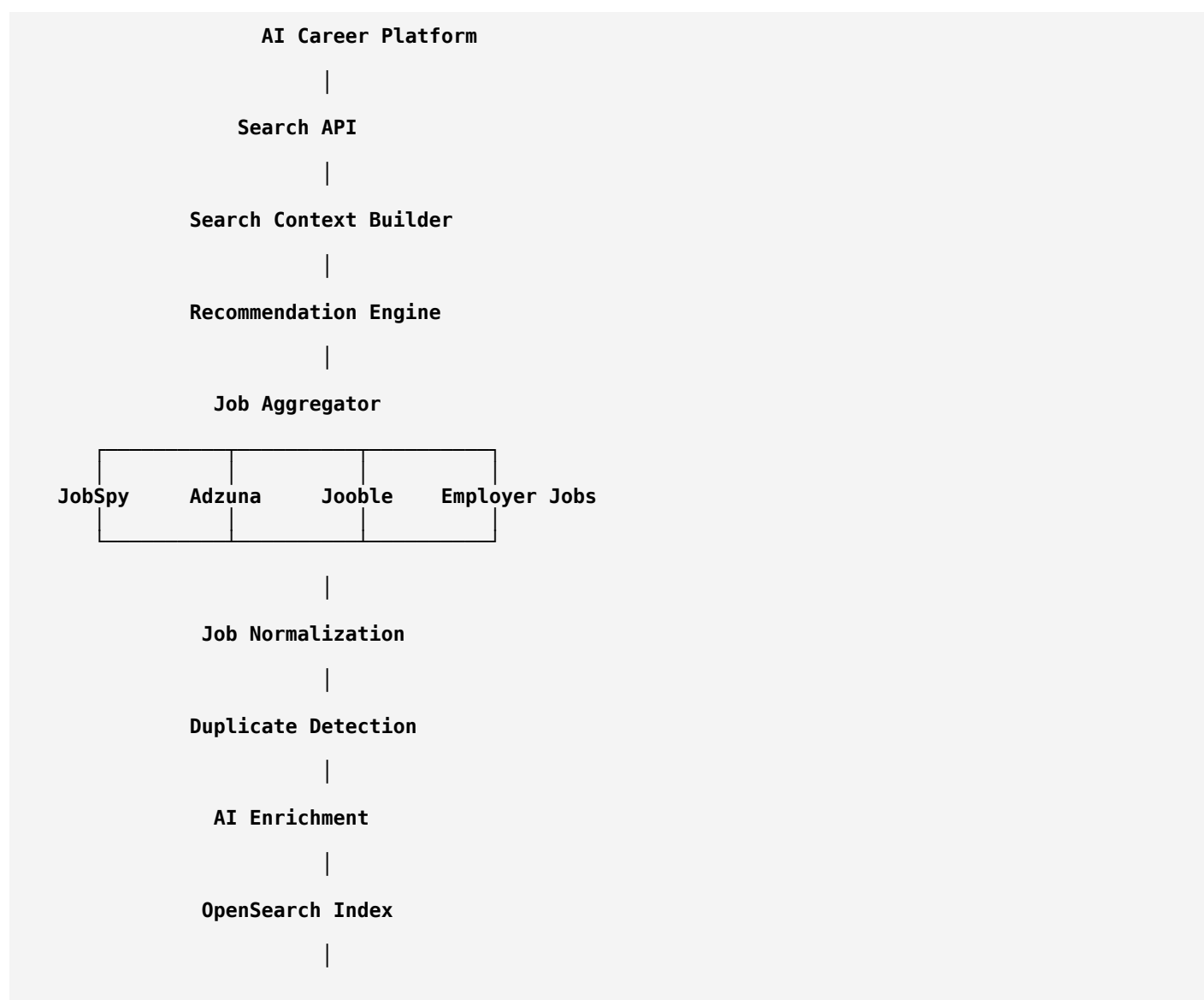
- LangChain (for orchestration ideas)
- LlamaIndex (RAG patterns)
- pgvector examples

My Biggest Recommendation

Don't think of JobSpy as **the backend**.

Think of it as **one plugin** in your system.

Your platform should look like this:



One more suggestion

Since you're already collecting repositories like **JobSpy**, **Career Ops**, **find-jobs**, and AI job search projects, I would create a dedicated **Provider Layer** from day one.

Treat every external integration (JobSpy, Jooble, Adzuna, employer feeds, future APIs) as interchangeable plugins behind a common interface. That way, if one provider breaks, changes its API, or is removed, you only update a single adapter instead of your entire search system. This design will save you a significant amount of maintenance as the platform grows.

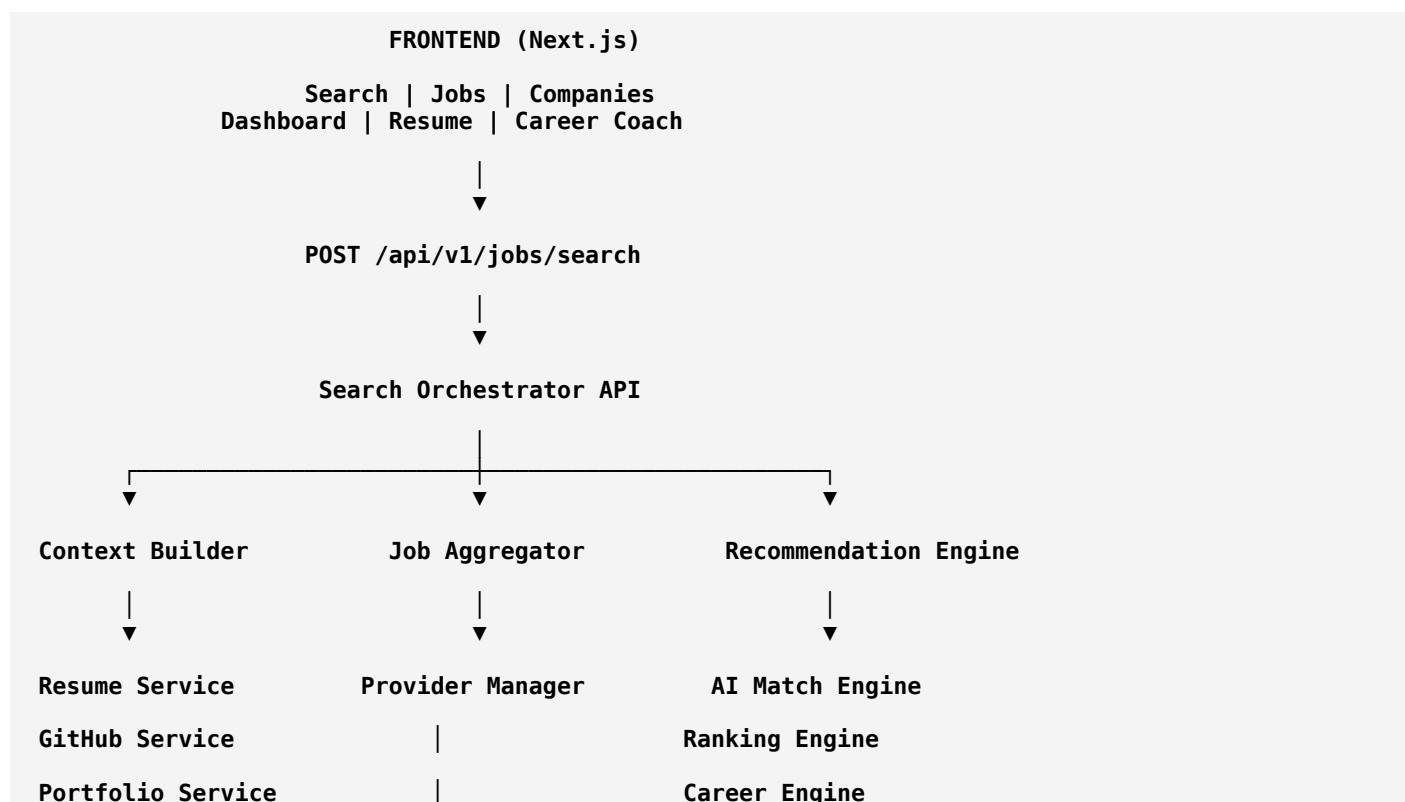
give me the final things please along with all the repository link

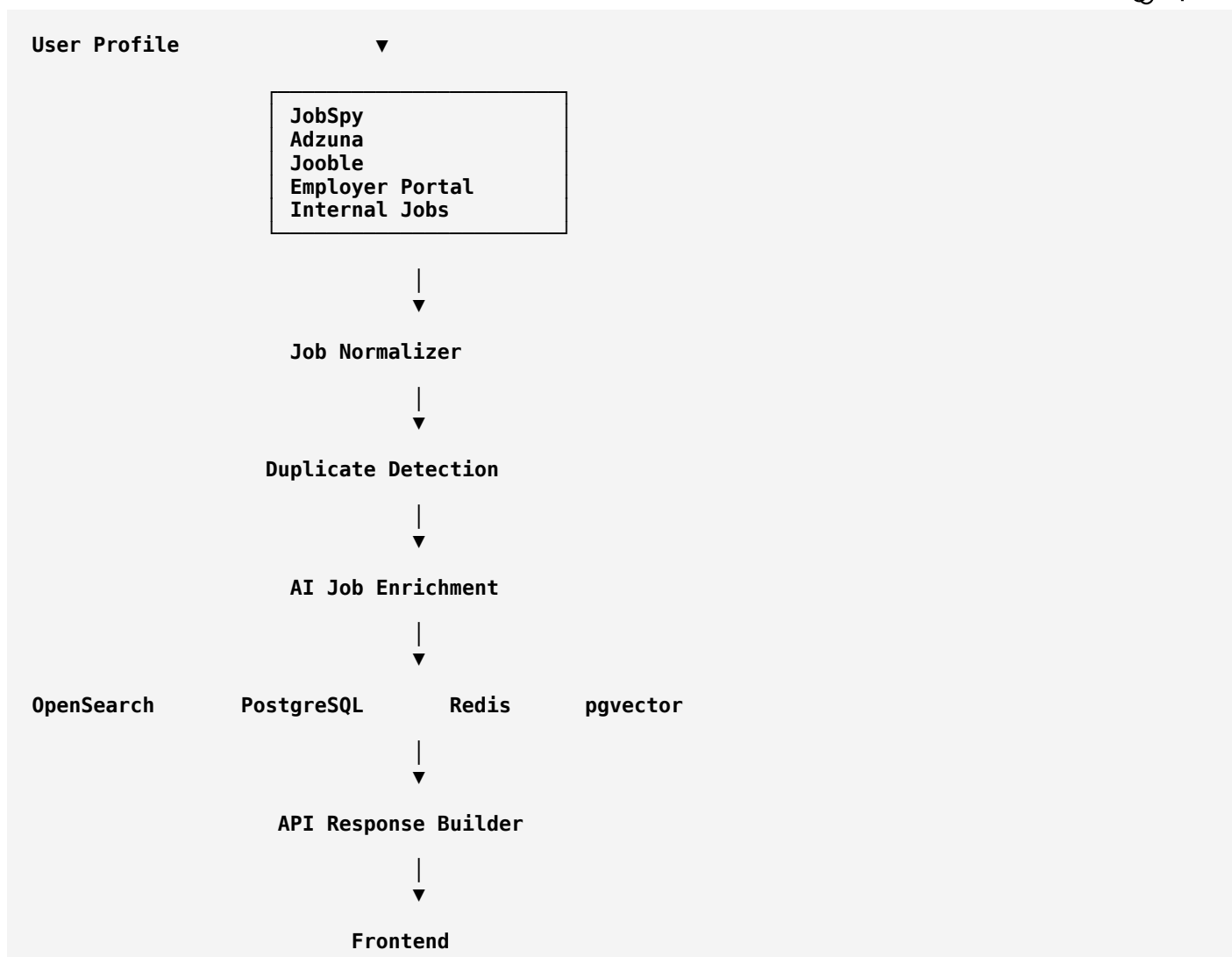
I think you've got something much bigger than a job portal here. If executed well, this could become an **AI Career Operating System**. The key is to **build your own intelligence layer** instead of depending on any one repository or provider.

Project Vision

Build an AI-powered Career Operating System that aggregates jobs from multiple sources, understands the user's profile (Resume + GitHub + Portfolio + Preferences), ranks opportunities using AI, and guides the user's entire career journey.

High-Level Architecture





Backend Modules

```

src/
modules/
  auth/
  users/
  jobs/
    controller/
    services/
      search.service
      context-builder.service
      aggregator.service
      provider-manager.service
      duplicate.service
      enrichment.service
      recommendation.service
      ranking.service
  
```


response-builder.service

providers/

jobspy/

adzuna/

jooble/

employer/

internal/

resume/

github/

portfolio/

ai/

companies/

salary/

notifications/

analytics/

skills/

employers/

applications/

career/

search/

admin/

shared/

config/

database/

Main API

Everything revolves around one endpoint.

POST /api/v1/jobs/search

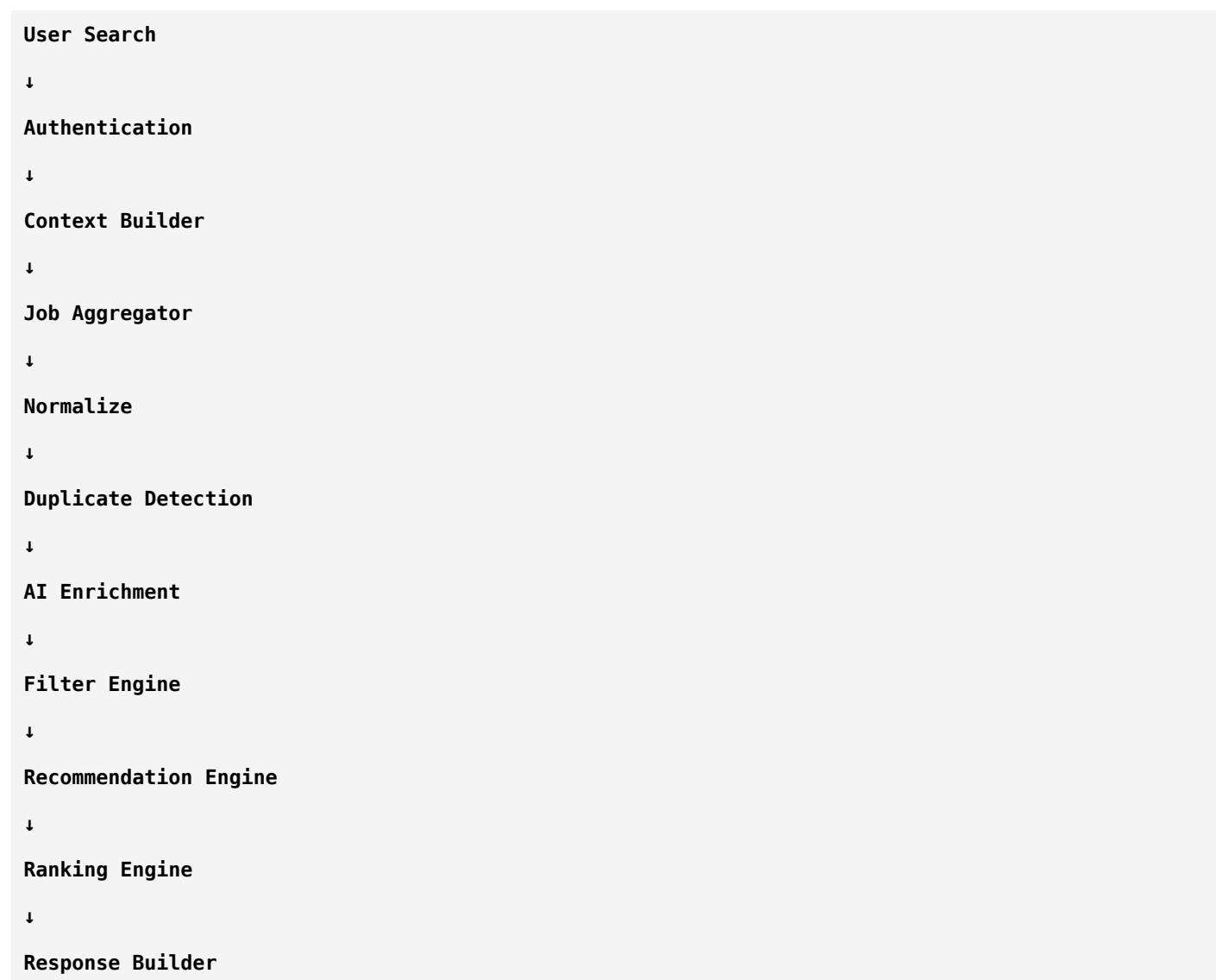
Frontend calls only this endpoint.

Backend handles:

- Authentication
- Profile
- Resume
- GitHub
- Portfolio

- Job Providers
- Search
- AI Ranking
- Recommendation
- Response

Search Pipeline



Search Context

Every search should build this internally.

```

TypeScript
SearchContext {
  query
  filters
  user
}
  
```

```

profile
resume
github
portfolio
careerGoals
preferences
searchHistory
savedJobs
appliedJobs
learningHistory
marketInsights
}

```

This becomes the heart of the platform.

AI Ranking

Every job gets scores.

```

Search Match
Resume Match
Profile Match
GitHub Match
Portfolio Match
Skill Match
Experience Match
Location Match
Salary Match
Career Goal Match
Popularity
Freshness
Company Score
Market Demand
Final AI Score

```

AI Explanation

Instead of

94%

Return

Why this matches

- ✓ React
- ✓ TypeScript
- ✓ Docker
- ✓ AWS

Missing

CI/CD

Kubernetes

Interview Probability

83%

AI Modules

- Resume Parser
- Resume ATS Score
- Resume Improvement
- GitHub Analyzer
- Portfolio Analyzer
- Skill Extractor
- Job Summarizer
- AI Ranking
- Recommendation Engine
- Career Coach
- Salary Prediction
- Skill Gap Detection
- Career Roadmap
- Interview Preparation

GitHub Intelligence

Analyze

- Languages
- Frameworks
- Libraries

- Architecture
- DevOps
- Cloud
- Databases
- Testing
- Commit Activity
- Open Source
- Repository Health

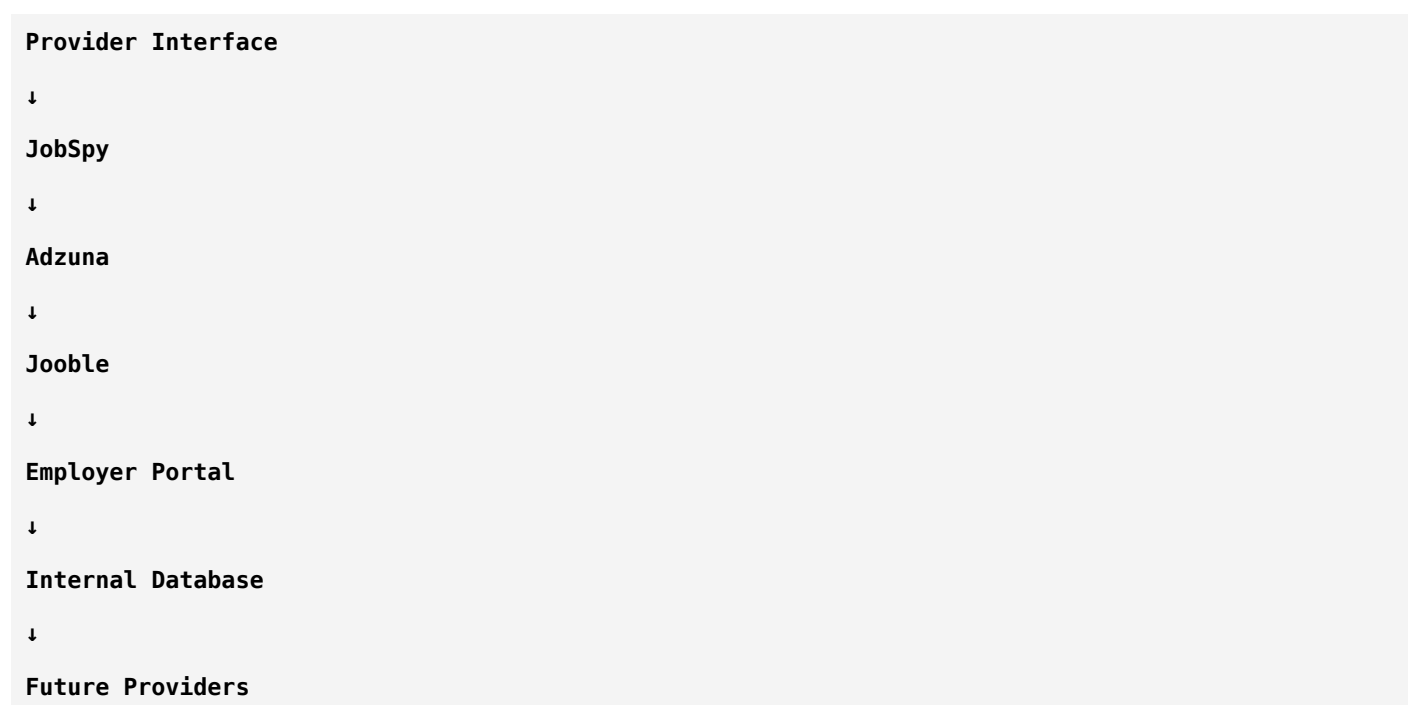
Gracefully handle:

- No GitHub
- Empty repositories
- Private repositories
- Fork-only profiles
- Inactive profiles

GitHub is an **optional enrichment layer**, never a requirement.

Job Aggregation

Providers should be plugins.



Never couple the application to one provider.

Database

Use

- PostgreSQL
- Prisma
- Redis
- OpenSearch
- pgvector

Queue

Use BullMQ

Events

```
Resume Uploaded
↓
Parse Resume
↓
Extract Skills
↓
Generate Embeddings
↓
Update Recommendations
↓
Notify User
```

Search Features

Standard Search

- Keyword
- Company
- Skills
- Technologies
- Industry
- Recruiter
- Natural Language

AI Search

- Semantic Search

- Intent Detection
- Query Expansion
- Query Correction
- AI Ranking
- Related Searches
- Similar Jobs

Filters

- Salary
 - Location
 - Remote
 - Hybrid
 - Visa
 - Relocation
 - Company
 - Benefits
 - Employment Type
 - Experience
 - Posted Date
 - Skills
 - Technologies
-

Candidate Features

- Resume Upload
- Resume Parsing
- Resume Optimization
- ATS Score
- GitHub Import
- Portfolio Import
- Saved Jobs
- Saved Searches
- Job Alerts
- One-click Apply (where supported)
- Application Tracker
- Career Dashboard

- AI Career Coach
 - Salary Insights
 - Company Reviews
 - Interview Preparation
 - Skill Gap Analysis
-

Employer Features

- Job Posting
 - Candidate Search
 - AI Candidate Ranking
 - ATS
 - Analytics
 - Team Management
 - Interview Scheduling
 - Offer Management
-

Admin Features

- User Management
 - Employer Management
 - Job Moderation
 - Fraud Detection
 - Analytics
 - Provider Health
 - API Monitoring
 - Search Monitoring
-

Tech Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- TanStack Query

Backend

- NestJS
- Prisma
- PostgreSQL
- Redis
- OpenSearch
- pgvector
- BullMQ

Infrastructure

- Docker
- Kubernetes
- GitHub Actions
- Prometheus
- Grafana

Repositories Worth Studying

Job Aggregation

JobSpy (Multi-provider scraping)

<https://github.com/speedyapply/JobSpy>

Find Jobs

<https://github.com/AnumQ/find-jobs>

Career Ops

<https://github.com/santifer/career-ops>

AI Powered Job Search

<https://github.com/maddykws/AI-Powered-Job-Search-Tool>

Resume

OpenResume

<https://github.com/xitanggg/open-resume>

Resume Matcher

<https://github.com/srbhr/Resume-Matcher>

Search

OpenSearch

<https://github.com/opensearch-project/OpenSearch>

Elasticsearch

<https://github.com/elastic/elasticsearch>

Meilisearch

<https://github.com/meilisearch/meilisearch>

AI / Vector Search

LangChain

<https://github.com/langchain-ai/langchain>

LlamaIndex

https://github.com/run-llama/llama_index

Qdrant

<https://github.com/qdrant/qdrant>

pgvector

<https://github.com/pgvector/pgvector>

GitHub Analysis

PyDriller

<https://github.com/ishepard/pydriller>

Gitleaks (security scanning ideas)

<https://github.com/gitleaks/gitleaks>

Background Jobs

BullMQ

<https://github.com/taskforcesh/bullmq>

API Documentation

Scalar

<https://github.com/scalar/scalar>

Swagger UI

<https://github.com/swagger-api/swagger-ui>

Development Roadmap

Phase 1 (MVP)

- Authentication
- User Profile
- Resume Upload
- Job Aggregation
- Search API
- Basic Filters
- Job Details
- Saved Jobs

Phase 2

- AI Resume Parsing
- Personalized Search Context
- Recommendation Engine
- AI Ranking
- Company Pages
- Salary Insights
- Search Analytics

Phase 3

- GitHub Integration
- Portfolio Analysis
- Skill Graph
- Career Dashboard
- AI Explanations
- Job Alerts
- Application Tracker

Phase 4

- AI Career Coach
- Learning Roadmap
- Employer AI
- One-click Apply (supported platforms)
- Multi-language Support
- Mobile Applications

Final Philosophy

The repositories above should be viewed as **building blocks**, not as your application.

- **JobSpy** → Collects jobs.
- **OpenResume** → Helps understand resumes.
- **OpenSearch** → Finds jobs efficiently.
- **pgvector/Qdrant** → Enables semantic search.
- **BullMQ** → Runs background processing.
- **LangChain/LlamaIndex** → Orchestrates AI workflows.

Your competitive advantage is the **Search Context Engine** and **AI Recommendation Layer** that sits on top of these components.

In other words, you're not building "another LinkedIn Jobs." You're building an **AI Career Intelligence Platform** where job aggregation is just the first step, and the real value comes from understanding the user, enriching the data, and delivering personalized, explainable career recommendations.

add all the things in an pdf and give me all the things in the pdf so that all the things are mentioned there and

Done! I've created a PDF containing a consolidated blueprint of the platform, including:

- Vision and goals
- Core feature set

- High-level architecture
- Search context design
- AI pipeline
- Backend modules
- Recommended tech stack
- Repository references
- Development roadmap

You can download it here:



One recommendation

The PDF above is a **condensed blueprint**. Based on everything we've discussed over the past conversations, I can also create a much more comprehensive document—around **150–300 pages**—that would resemble a real Software Requirements Specification (SRS) and technical architecture document.

It would include:

- Executive Summary
- Product Vision
- Functional Requirements
- Non-Functional Requirements
- Complete System Architecture
- Low-Level Design (LLD)
- High-Level Design (HLD)
- End-to-End User Flows
- Database Design (with Prisma models and ER diagrams)
- API Specifications (every endpoint)
- Search Engine Architecture
- AI Recommendation Engine
- Resume Parsing Pipeline
- GitHub Analysis Engine
- Portfolio Analysis Engine
- Job Aggregation Engine
- Search Context Engine
- Ranking Algorithm
- OpenSearch Architecture
- Redis Caching Strategy

- BullMQ Queue Design
- Notification System
- Authentication & RBAC
- Employer Portal
- Admin Portal
- Candidate Dashboard
- Frontend UI Wireframes
- Backend Folder Structure
- Microservice Migration Plan
- Deployment Architecture (Docker/Kubernetes)
- Security Best Practices
- Performance & Scalability
- Monitoring & Logging
- Testing Strategy
- CI/CD Pipeline
- Phase-wise Development Plan
- Repository references with explanations
- Future AI features and roadmap

That document would be suitable for handing to a full engineering team (frontend, backend, AI/ML, DevOps, QA, and UI/UX) to build the platform from scratch.